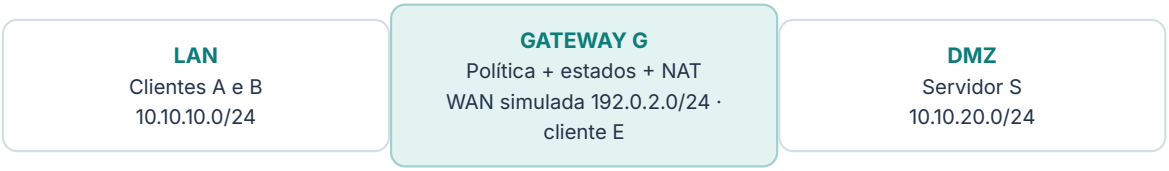
INFRAESTRUTURA · SUPORTE · SEGURANÇA

FIREWALLS

Tipos, funcionamento, operação e investigação: compreender decisões de filtragem, configurar políticas e investigar falhas com evidências.



EXECUTADO NESTA EDIÇÃO

Laboratórios Linux em redes isoladas (namespaces): host, encaminhamento, estados, DNAT, rastreamento, retorno assimétrico e reversão, com contadores, logs, conntrack e capturas reais

PREPARADO PARA A VM

Windows Firewall e WFP com três scripts PowerShell (sintaxe validada) e roteiro pfSense; 6 políticas nftables e 12 capturas anexadas

Apostila de consulta e laboratório • Outubro de 2026 • Fontes: NIST, Netgate, Netfilter, Microsoft, AWS, Palo Alto Networks, RFC Editor, Wireshark

Exemplos de configuração são de laboratório e não constituem política pronta para produção. Exercícios em redes isoladas, com acesso por console e dados fictícios.

EDIÇÃO 2 / 30 DE OUTUBRO DE 2026 / 55 PÁGINAS

Como utilizar esta apostila

A pergunta que orienta tudo: **qual comunicação, entre quais pontos, falhou em qual horário, e qual evidência localiza a decisão?** Uma mensagem de timeout descreve um sintoma; não identifica, sozinha, o equipamento responsável.

Rotas de leitura

Preciso...	Começar por...
Entender os tipos	Fases 1 e 2 (capítulos 1 a 10).
Entender uma decisão	Caminho, precedência, estados e NAT (capítulos 11 a 15).
Operar Windows e Linux	Fase 4 (capítulos 18 a 24).
Praticar	Laboratórios (capítulos 25 a 34); os Linux vêm com saídas reais.
Investigar uma falha	Casos (capítulos 35 a 38) e o método abaixo.
Documentar e revisar	Projeto (39 e 40) e apêndices.

Método

01 Delimite a transação e identifique o endereço efetivamente usado.

02 Desenhe o caminho de ida e o de retorno, incluindo túneis e traduções.

03 Confirme o serviço em escuta e a política efetiva.

04 Compare captura de entrada, captura de saída, estado e log.

05 Mude uma condição por vez e repita o teste.

06 Documente a conclusão, a reversão e o que permanece incerto.

Convenções

- EXECUTADO marca saídas reais desta edição. Os laboratórios Linux rodaram em Linux 6.18, nftables 1.0.9 e conntrack 1.4.8, numa topologia de **namespaces de rede**: cada máquina (A, B, G, S, E) é uma pilha de rede isolada do mesmo kernel, ligada às outras por bridges virtuais. É tráfego real entre essas pilhas, sem contato com rede física (capítulo 25).
- NÃO EXECUTADO marca Windows, pfSense e o laboratório IPv6 (o kernel do ambiente de produção desta edição não tem IPv6). Os scripts PowerShell tiveram só a sintaxe validada.
- CORREÇÃO marca pontos corrigidos em relação à Edição 1 (Apêndice G). NOVO marca acréscimos.
- "Esperado" significa previsão para a topologia descrita. Nos laboratórios executados, o resultado observado aparece logo abaixo e é ele que vale. Nas saídas, `[…]` indica linha encurtada pelo próprio script de execução.
- Na variante Linux, as interfaces do gateway se chamam `lan0`, `dmz0` e `wan0` (na Edição 1, `ens18` e `ens19`). Substitua pelos nomes reais da sua VM.

Anexo

O PDF incorpora `laboratorio_firewalls.zip`: topologia e execução dos laboratórios Linux, as seis políticas nftables, os scripts Windows, o roteiro pfSense e as 12 capturas reais com SHA256. Extraia pelo painel de anexos do leitor ou com `pdfdetach -saveall`.

Sumário

FASE 1 · FUNDAMENTOS

01 O que um firewall decide 05
02 Descrever um fluxo corretamente 05
03 Políticas, ações e testes 07

FASE 2 · TIPOS

04 Classificar sem misturar conceitos 08
05 Filtragem stateless 08
06 Inspeção stateful 09
07 Gateway de circuito e proxy 09
08 NGFW e identificação de aplicações 09
09 WAF: proteção da aplicação web 11
10 Host, rede e controles em nuvem 11

FASE 3 · FUNCIONAMENTO INTERNO

11 Caminho do pacote no Linux 12
12 Precedência: três modelos 12
13 Estados, expiração e sessões antigas 14
14 TCP, UDP e ICMP na decisão 14
15 NAT: tradução e permissão 16
16 IPv6 e descoberta de MTU 17
17 TLS, inspeção e visibilidade 17

FASE 4 · OPERAÇÃO

18 Windows: inventário antes de alterar 18
19 Windows: criar, registrar e remover 18
20 Windows: evidência no Filtering Platform 20
21 Linux: reconhecer o responsável 20
22 Linux: regras legíveis e observáveis 22
23 Borda: operar pelo caminho real 23
24 Mudança, backup e reversão 24

PRÁTICA

25 Preparar o laboratório isolado 25
26 Laboratório 1: serviço e linha de base 25
27 Laboratório 2: política de host Linux 27
28 Laboratório 3: firewall do Windows 28
29 Laboratório 4: acompanhar uma sessão 29
30 Laboratório 5: encaminhamento Linux 29
31 Laboratório 6: publicar na DMZ 31
32 Laboratório 7: provar a hipótese 31
33 Laboratório 8: comparar IPv4 e IPv6 33
34 Laboratório 9: ensaiar a reversão 33

FASE 5 · INVESTIGAÇÃO E PROJETO

35 Caso 1: a regra existe, mas falha 34
36 Caso 2: DNAT correto, retorno errado 34
37 Caso 3: por IP funciona, por nome não 36
38 Caso 4: domínio e e-mail 36
39 Projetar zonas e dependências 38
40 Alta disponibilidade e capacidade 38

APÊNDICES

A Exercícios 40
B Gabarito comentado 41

C Checklist e registro de mudança 42
D Comandos e glossário 43
E Scripts do laboratório 44
F Referências oficiais 53
G Registro de edições e verificação 54

01 O que um firewall decide

Um firewall aplica uma política ao tráfego que consegue observar. Pode controlar a comunicação entre redes ou a de um próprio computador. Seu valor depende tanto da posição na rede quanto da qualidade das regras [1].

> **PERMISSÃO É CONDIÇÃO, NÃO SELO** Autorizar o cliente a alcançar TCP/443 permite estabelecer determinada comunicação. Não demonstra que o usuário é legítimo, que o servidor está corrigido ou que o conteúdo é seguro. Outros controles verificam essas propriedades.

Considere um servidor de chamados: usuários acessam a aplicação, a equipe técnica o administra e o servidor consulta DNS. São três necessidades distintas. Uma regra "qualquer origem, qualquer destino, qualquer serviço" esconde essas diferenças e impede uma revisão útil.

Pergunta	Decisão de projeto
Quem inicia?	Estações, servidor, administrador ou sistema externo
Quem recebe?	Endereço e serviço realmente utilizados
O que é necessário?	Protocolo, porta e, se suportado, aplicação
Onde controlar?	Host, ponto de roteamento, proxy ou serviço em nuvem
Como comprovar?	Teste positivo, teste negativo, log e captura

Limite físico e lógico

Se dois computadores trocam quadros no mesmo segmento sem passar pelo firewall de borda, uma regra nessa borda não controla essa conversa. Coloque o controle no caminho: segmentação roteada, controle distribuído ou firewall nos hosts. Um firewall não bloqueia um pacote que nunca passa por seu mecanismo de inspeção: primeiro valide o caminho, depois avalie a regra.

02 Descrever um fluxo corretamente

Uma conexão de cliente normalmente usa porta de origem efêmera e porta de destino do serviço. No retorno, endereços e portas se invertem.

Trecho	Origem	Destino
Solicitação TCP	10.10.10.10:53024	10.10.20.20:443
Resposta TCP	10.10.20.20:443	10.10.10.10:53024

Protocolo, IP e porta de origem, IP e porta de destino formam a 5-tupla. O ICMP usa tipo e código; não tem portas TCP/UDP. O `conntrack -L` mostra exatamente as duas tuplas de cada fluxo, a original e a de retorno (capítulos 13 e 15).

Direção depende do observador

Para a estação, a solicitação é saída; para o servidor, entrada; para o roteador Linux entre eles, encaminhamento. No pfSense, uma regra comum na aba de interface inspeciona a entrada naquela interface: tráfego originado na LAN é tratado na entrada da LAN [2].

> **PEDIDO BEM FORMADO** "Permitir que 10.10.10.10 inicie TCP para 10.10.20.20 na porta 443, para consultar o portal interno; responsável: infraestrutura; validade: permanente com revisão trimestral." Acrescente endereço antes e depois do NAT, zona, família IP e ponto de aplicação. "Liberar HTTPS" deixa origem, destino e escopo indefinidos.

No firewall do servidor, o campo de porta local corresponde a 443 nesse exemplo; no do cliente, uma regra de saída usa a porta remota 443. Confundir esses campos cria regras que não correspondem ao fluxo.

03 Políticas, ações e testes

A política padrão define o tratamento quando nenhuma exceção decide o tráfego. Projetos de acesso mínimo descrevem permissões específicas e negam o restante no escopo controlado, preservando dependências e administração.

Ação	Efeito usual	O que o cliente percebe
Allow / accept	Permite prosseguir naquele estágio	Ainda pode falhar adiante
Drop	Descarta sem resposta	Espera e retransmissões
Reject	Descarta e tenta informar a rejeição	Erro rápido, conforme o protocolo
Log	Registra uma observação	Não implica, sozinho, permitir ou negar

Drop, reject e porta fechada, lado a lado NOVO

EXECUTADO Cliente B contra S, com `lab-host.nft` (drop), com `lab-host-reject.nft` (reject com RST) e sem firewall numa porta sem serviço:

```
$ B contra S com a politica que DESCARTA (lab-host.nft)
B    http://10.10.20.20:8080/           -> HTTP 000  tempo 3.002511s  curl_exit=28
$ B contra S com a politica que REJEITA com RST (lab-host-reject.nft)
B    http://10.10.20.20:8080/           -> HTTP 000  tempo 0.000135s  curl_exit=7
1 2026-10-30 15:42:35.843698957  10.10.10.11 → 10.10.20.20  TCP 74 47194 → 8080 [SYN] Seq=1097066448
Win=64240 Len
2 2026-10-30 15:42:35.843715936  10.10.20.20 → 10.10.10.11  TCP 54 8080 → 47194 [RST, ACK] Seq=0
Ack=1097066449 Wi

$ B contra S SEM firewall e SEM servico na porta 9999
B    http://10.10.20.20:9999/           -> HTTP 000  tempo 0.000171s  curl_exit=7
```

O drop custou 3 s (o limite do curl, `curl_exit=28`). O reject e a porta fechada produziram o **mesmo** resultado no cliente: recusa imediata (`curl_exit=7`) e um RST/ACK vindo do endereço de S. Ou seja, **um RST não prova reject do firewall, e um timeout não prova drop**: serviço parado, rota errada e intermediários produzem sintomas iguais. O que separa as hipóteses é log, contador e captura nos pontos certos.

Política de entrada não é política de saída

Restrinja serviços recebidos e também conexões iniciadas pelos servidores. Documente DNS, horário, atualização e integrações antes de restringir a saída.

Validar inclui demonstrar o bloqueio

Permitida A → S:8080, verifique que A funciona, que B não funciona e que outra porta continua inacessível; confirme que administração e resolução de nomes seguem operantes. Uma regra com contador zero pode estar errada, obsoleta ou só não ter recebido tráfego na janela: isso não autoriza excluí-la automaticamente.

04 Classificar sem misturar conceitos

Há perguntas diferentes: como inspeciona, onde atua e como é entregue. Um appliance pode executar inspeção stateful e de aplicação; uma VM pode oferecer mecanismos semelhantes; um firewall de host pode acompanhar estados.

Eixo	Categorias	O que define
Mecanismo	Stateless, stateful, proxy, inspeção de aplicação	As informações usadas na decisão
Posição	Host, borda, segmentação interna, frente da aplicação	O tráfego que consegue observar
Entrega	Appliance, software, VM, serviço gerenciado	Implantação e responsabilidades
Especialização	WAF, controle de aplicações, filtragem DNS	O problema específico tratado

NGFW e UTM são denominações de conjuntos de recursos. Verifique identificação de aplicações, inspeção TLS, prevenção de intrusão, atualização de assinaturas, capacidade e visibilidade dos logs: o mesmo nome comercial não garante o mesmo comportamento.

Rede e host se complementam. Um filtro entre VLANs restringe o acesso ao servidor; o firewall do servidor limita a regra a um programa. Um bloqueio em qualquer ponto ainda impede a operação, e a investigação deve considerar os dois.

Exercício: "firewall virtual stateful na borda" descreve três propriedades compatíveis. "Firewall na nuvem é sempre stateless" não é classificação válida: depende do controle. Security groups e network ACLs da AWS são mecanismos diferentes [3, 4].

05 Filtragem stateless

A filtragem sem estado avalia pacotes por campos e regras, sem manter o histórico da conversa para autorizar a resposta. É útil para controles simples e explícitos, mas exige pensar nas duas direções.

Exemplo. A inicia TCP de 10.10.10.10:53024 para S:443 e uma ACL permite esse sentido. A resposta sai de 443 e chega à porta 53024 de A: a ACL precisa permitir esse retorno; repetir "destino 443" no retorno não corresponde ao pacote. Uma regra baseada na flag ACK inspeciona um campo; não reconstrói uma conversa autorizada.

Aspecto	Leitura operacional
Vantagem	Condições explícitas, úteis em filtros de rede bem delimitados
Custo	Regras de retorno e faixas efêmeras precisam ser consideradas
Limitação	Não vincula cada resposta a uma solicitação permitida
Evidência	Regra correspondente, contador, sentido e captura

Na AWS, network ACLs atuam na subnet, permitem allow e deny, avaliam regras numeradas em ordem crescente até a primeira correspondência e são stateless. Não filtram o tráfego roteado dentro da mesma subnet e têm exceções documentadas [4].

Prática de papel: escreva as duas 5-tuplas de uma consulta DNS UDP e explique por que "destino UDP/53 nos dois sentidos" não descreve a resposta ao cliente.

06 Inspeção stateful

Um mecanismo stateful mantém informações sobre fluxos e usa esse contexto nas decisões. O primeiro tráfego autorizado pode criar um estado; os pacotes seguintes são comparados a ele. Uma entrada na tabela demonstra que o mecanismo registrou o fluxo, não que a aplicação respondeu corretamente.

Momento	Pergunta do operador
Início	A política autoriza uma nova comunicação?
Continuidade	O pacote corresponde a um fluxo acompanhado?
Inatividade	O estado ainda existe ou expirou?
Mudança de regra	A sessão antiga continua válida? O produto a reavalia? (capítulo 29)
Falha de nó	O novo nó recebeu o estado necessário?

UDP também tem acompanhamento. Sem handshake, o firewall ainda pode acompanhar tuplas, direções e tempo. "Sem conexão" no transporte não significa "impossível criar estado no firewall".

Teste de raciocínio. Uma aplicação funciona, fica ociosa e falha. Hipóteses: expiração de estado, tradução NAT expirada, sessão da aplicação ou intermediário. Registre o intervalo e repita com captura; aumentar todos os timeouts sem evidência pode só esconder o problema. No Netfilter, as expressões `ct` usam o connection tracking: new, established, related e invalid não são estados de negócio da aplicação [5].

07 Gateway de circuito e proxy

Um gateway de circuito intermedeia a criação de conexões e pode exigir autorização antes do acesso. No SOCKS5, o cliente negocia um método e solicita uma operação de relay; isso não implica interpretar HTTP nem corrigir a aplicação [6]. Um proxy de aplicação decide por destino, método, cabeçalhos ou autenticação, conforme produto e configuração. Um túnel HTTP CONNECT sem inspeção TLS não expõe o conteúdo cifrado.

Modelo	Quem utiliza	Questão operacional
Forward proxy	Clientes acessando destinos	O cliente está configurado para usar o proxy?
Reverse proxy	Clientes acessando um serviço publicado	O backend aceita acesso direto que contorne o proxy?
Gateway de circuito	Clientes que negociam relay	Qual destino foi solicitado e autorizado?

Duas conexões, duas investigações. Se o intermediário termina a conexão, há os trechos cliente → proxy e proxy → servidor. O primeiro pode funcionar enquanto o segundo falha por DNS, rota, certificado ou política: correlacione horários e identificadores de requisição. Um erro HTTP gerado pelo proxy prova que o cliente alcançou o proxy, não que o proxy alcançou o servidor. Defina destinos, autenticação, retenção e proteção da administração; um relay irrestrito passa a iniciar conexões em nome de qualquer cliente.

08 NGFW e identificação de aplicações

Um firewall de próxima geração combina acompanhamento de sessões com identificação de aplicações e inspeções adicionais; o que está ativo depende de plataforma, licença e política. TCP/443 é condição de transporte e não identifica sozinha a aplicação; a classificação pode amadurecer durante a sessão.

Como exemplo específico, o App-ID da Palo Alto Networks combina técnicas de classificação e pode reaplicar assinaturas ao fluxo decifrado; a opção application-default relaciona a aplicação às portas padrão do produto. Não generalize esses detalhes para outras marcas [7, 8].

01 Traduza o pedido de negócio em origem, destino e aplicações necessárias.

02 Observe como o tráfego legítimo é classificado antes de restringir.

03 Verifique dependências: autenticação, atualização, redirecionamento e CDN.

04 Defina o tratamento de aplicações desconhecidas e de tráfego não inspecionado.

05 Teste sessões novas e acompanhe os registros de decisão (regra e ação, aplicação identificada, inspeção aplicada, motivo de encerramento, versão de conteúdo).

NÃO PRESUMA INSPEÇÃO Uma sessão permitida não significa que IPS, antivírus ou inspeção TLS foram aplicados. Confirme os perfis vinculados e o resultado registrado para aquela sessão.

09 WAF: proteção da aplicação web

Um WAF aplica políticas a requisições web e pode detectar padrões associados a ataques, conforme as regras e o produto. No AWS WAF, as ações incluem permitir, bloquear e contar; o modo Count observa correspondências antes de mudar o tratamento, e o serviço atua sobre recursos associados [9].

Exercício com tráfego benigno. Crie uma regra que reconheça o cabeçalho fictício `X-Lab-Test: revisar`. Primeiro conte; depois bloqueie só esse critério. Envie uma requisição com o cabeçalho e outra sem. O objetivo é ver a decisão de camada de aplicação sem usar exploração real.

Pergunta	Verificação
O HTTP chegou ao WAF?	Registro correspondente e recurso protegido
A regra bloqueou?	Identificador da regra e ação efetiva
Foi o WAF ou a aplicação?	Registro do WAF × log do backend
Há caminho alternativo?	Acesso direto ao endereço de origem

Para falso positivo num formulário, reduza a exceção ao campo e ao caminho necessários; desligar um conjunto inteiro de proteções esconde o erro e amplia a exposição. WAF não substitui firewall de host, autenticação, autorização ou correção do código, e um HTTP 403 pode vir da aplicação.

10 Host, rede e controles em nuvem

Controle	Escopo didático	Atenção
Windows Firewall	Entrada e saída do servidor	Perfil e política efetiva
Firewall entre VLANs	Fluxos entre segmentos	Tráfego no mesmo segmento pode não cruzá-lo
AWS security group	Recursos e interfaces associados	Stateful; só regras de permissão
AWS network ACL	Entrada e saída de subnet	Stateless; allow/deny e retorno explícito
WAF	Requisições web do recurso protegido	Depende de associação e caminho

Security groups permitem o tráfego de resposta das comunicações autorizadas, e há tráfego de infraestrutura que esses controles não filtram, como certas comunicações com serviços reservados da VPC [3]. Antes de abrir uma regra na nuvem: o recurso está no grupo correto? O IP público é do recurso ou do balanceador? A rota aponta para o componente esperado? O backend vê o IP do cliente ou do proxy?

Resultado da fase: descreva a solução sem depender da marca ("controle stateful entre segmentos, firewall nos hosts e WAF na publicação web") e diga qual evidência cada camada produz.

11 Caminho do pacote no Linux



Pacotes criados pelo próprio host passam por OUTPUT e depois POSTROUTING. Modelo simplificado para IP roteado: bridge, netdev, flow offload e namespaces podem mudar os pontos relevantes.

No nftables, uma base chain conecta regras a um hook; uma cadeia regular só participa se for chamada; prioridades ordenam base chains no mesmo hook [10].

O erro recorrente, executado

EXECUTADO Em G, libera-se INPUT TCP/8080 e o forward fica com policy drop (`lab-gateway-input-errado.nft`); depois, a política correta no forward (`lab-router.nft`):

```
$ nft -f lab-gateway-input-errado.nft   (em G: libera INPUT 8080, forward drop)
A    http://10.10.20.20:8080/           -> HTTP 000  tempo 3.002646s  curl_exit=28
                tcp dport 8080 counter packets 0 bytes 0 accept comment "LAB: regra no lugar errado"
                counter packets 3 bytes 180 comment "LAB: tudo que chega ao forward"

$ nft -c -f lab-router.nft && nft -f lab-router.nft   (em G)
A    http://10.10.20.20:8080/           -> HTTP 200  tempo 0.001177s  curl_exit=0
B    http://10.10.20.20:8080/           -> HTTP 000  tempo 3.002452s  curl_exit=28

$ contadores de lab_router
                ct state established,related counter packets 11 bytes 861 accept
                iifname "lan0" oifname "dmz0" ip saddr 10.10.10.10 ip daddr 10.10.20.20 tcp dport 8080
counter packets 1 bytes 60 accept comment "LAB: A para S:8080"
                counter packets 3 bytes 180 drop

$ dmesg | grep LAB_FWD_DROP
LAB_FWD_DROP IN=lan0 OUT=dmz0 MAC=b6:67:c1:28:a3:d1:a2:bf:55:0e:8c:ec:08:00 SRC=10.10.10.11
DST=10.10.20.20 LEN=60 TOS=0x00 PREC=0x00 TTL=63 ID=59025  […]
LAB_FWD_DROP IN=lan0 OUT=dmz0 MAC=b6:67:c1:28:a3:d1:a2:bf:55:0e:8c:ec:08:00 SRC=10.10.10.11
DST=10.10.20.20 LEN=60 TOS=0x00 PREC=0x00 TTL=63 ID=59026  […]

$ SYN de B (10.10.10.11) em cada interface de G
lan0: 3 quadro(s)   dmz0: 0 quadro(s)
SYN de A: lan0 1, dmz0 1
```

A regra de INPUT terminou com **0 pacotes**: o acesso de A a S nunca passa por INPUT, porque o destino não é o gateway. Os 3 pacotes (SYN e duas retransmissões) foram contados no forward. Com a regra no lugar certo, A recebe HTTP 200 e B é descartado, e as capturas nas duas interfaces de G localizam a decisão: os SYN de B entram por `lan0` e não saem por `dmz0`.

12 Precedência: três modelos

Não transporte a regra mental de uma plataforma para outra. "Está acima na tela" só explica a decisão se o mecanismo usar aquela ordem.

Plataforma	Modelo a verificar	Erro a evitar
Windows Firewall	Sem ordenação definida pelo administrador. Bloqueio explícito prevalece sobre permissão conflitante; fora disso, a regra mais específica prevalece; sem permissão, vale o padrão [11]. NOVO	Tentar resolver conflito movendo a regra na lista ou criando outra permissão mais específica contra um bloqueio
nftables	Sequência na cadeia, saltos, retornos, hooks e prioridades	Achar que um accept impede todo processamento posterior
pfSense	Regras de interface, grupos, floating (com ou sem *quick*) e regras automáticas	Examinar só a aba da interface

A Edição 1 resumia o Windows como "sem ordenação por peso", correto, mas omitia que a especificidade decide entre permissões quando não há bloqueio explícito. Cenários autenticados de bypass com IPsec exigem análise específica. No nftables, accept numa base chain ainda pode encontrar um drop em outra base chain posterior; drop encerra o tratamento [12].

Estudo de papel: no Windows, allow para 10.10.10.10 e block para 10.10.10.0/24 na mesma porta. O allow mais específico não anula o bloqueio explícito. Restrinja ou remova o conflito conforme a política pretendida; não adicione uma terceira permissão. `inventario_firewall.ps1` (capítulo 18) avisa quando há bloqueio explícito para a porta.

Registro mínimo: exportação da política, identificador da regra efetiva, interface, direção, origem, destino e horário. Uma captura de tela isolada de uma regra sustenta conclusões frágeis.

13 Estados, expiração e sessões antigas

Estado (ct)	Interpretação útil
new	Tráfego de início ou ainda sem resposta observada; não significa apenas SYN
established	Fluxo que já tem comunicação reconhecida nas duas direções
related	Novo fluxo ou pacote associado a outro, como certos erros ICMP
invalid	Não associado validamente pelo acompanhamento naquele contexto

Related não significa "qualquer coisa parecida com a conexão": pode depender de metadados e helpers. Não habilite helpers indiscriminadamente [5]. O estado do firewall também não é o estado TCP da aplicação: o servidor pode ter fechado o socket enquanto o intermediário ainda guarda a entrada.

Ver os estados no Linux NOVO

O `conntrack` é a interface de usuário do connection tracking do kernel: `-L` lista, `-E` mostra eventos em tempo real e `-D` remove entradas com filtros como `-p`, `-s` e `--dport`. `-F` esvazia a tabela inteira [37].

```
sudo conntrack -L -p tcp --dport 8080          # leitura
sudo conntrack -E -p tcp --dport 8080          # leitura, em tempo real
sudo conntrack -D -p tcp -s 10.10.10.10 --dport 8080   # MUDANÇA: só o fluxo do teste
```

No pfSense, Diagnostics → States mostra o equivalente, inclusive endereços traduzidos [13]. O capítulo 29 executa o experimento "regra nova, conexão antiga".

> **NÃO LIMPE TUDO COMO PRIMEIRO TESTE** Esvaziar a tabela interrompe tráfego legítimo e destrói a evidência da falha. Em produção, trate remoção de estados como mudança com impacto explícito; no laboratório, filtre o fluxo ensaiado.

14 TCP, UDP e ICMP na decisão

Uma abertura TCP típica envolve SYN, SYN/ACK e ACK. Concluir o handshake demonstra conectividade TCP naquele instante, não autenticação nem sucesso da aplicação [14].

Observação	Hipóteses	Próxima evidência
SYN repetido, sem resposta	Drop, perda, rota, host ausente	Captura de entrada e saída e no servidor
RST após SYN	Porta sem listener ou reject (capítulo 3)	Emissor real, sockets e logs
Handshake completo; HTTP 403	Controle de aplicação ou WAF	Logs de requisição
UDP sem resposta	Serviço, filtragem, perda ou silêncio normal	Solicitação válida e log do serviço
ICMP unreachable	Falha de entrega indicada	Tipo, código e pacote citado

UDP "aberta" é difícil de provar: um datagrama arbitrário pode não gerar resposta mesmo com serviço ativo; use uma solicitação válida do protocolo. **Bloquear todo ICMP** transforma falhas explicáveis em esperas silenciosas e prejudica a descoberta de MTU.

Exercício: o servidor recebe o SYN e envia SYN/ACK, mas o cliente não o observa. A aplicação ainda não é a suspeita; priorize o retorno. O capítulo 36 mostra esse caso executado.

15 NAT: tradução e permissão

SNAT altera a origem; DNAT altera o destino; PAT também traduz portas. Tradução resolve endereçamento e publicação, mas não substitui a política de autorização. No NAT stateful do nftables, a primeira decisão estabelece o mapeamento usado pelos pacotes seguintes; masquerade é SNAT pelo endereço da interface de saída [15].

EXECUTADO Publicação de S:8080 em 192.0.2.1:18080, só para E (`lab-nat.nft`). A mesma conversa vista na WAN e na DMZ de G, e a entrada do conntrack:

```
$ nft -f lab-nat.nft   (em G)
E    http://192.0.2.1:18080/            -> HTTP 200  tempo 0.001359s  curl_exit=0
E2   http://192.0.2.1:18080/            -> HTTP 000  tempo 3.002987s  curl_exit=28

$ conntrack -L -p tcp --dport 18080   (em G: tupla original e de retorno)
tcp      6 113 TIME_WAIT src=192.0.2.10 dst=192.0.2.1 sport=36154 dport=18080 src=10.10.20.20
dst=192.0.2.10 sport=8080 dport=36154 [ASSURED]

$ a mesma conversa de E vista em wan0 e dmz0 (primeiros quadros)
wan0:
3 2026-10-30 15:43:09.384105637   192.0.2.10 → 192.0.2.1    TCP 74 36154 → 18080 [SYN] Seq=196952930
Win=64240 Len
4 2026-10-30 15:43:09.384146000    192.0.2.1 → 192.0.2.10   TCP 74 18080 → 36154 [SYN, ACK]
Seq=1800147763 Ack=196
dmz0:
1 2026-10-30 15:43:09.384124720   192.0.2.10 → 10.10.20.20  TCP 74 36154 → 8080 [SYN] Seq=196952930
Win=64240 Len=
2 2026-10-30 15:43:09.384142560  10.10.20.20 → 192.0.2.10   TCP 74 8080 → 36154 [SYN, ACK]
Seq=1800147763 Ack=1969

$ contadores
iifname "wan0" tcp dport 18080 counter packets 4 bytes 240 dnat to 10.10.20.20:8080 comment "LAB-DNAT-
HTTP"
ct state established,related counter packets 12 bytes 912 accept
iifname "wan0" oifname "dmz0" ip saddr 192.0.2.10 ip daddr 10.10.20.20 tcp dport 8080 ct status dnat
counter packets 1 bytes 60 accept comment "LAB: E publicado"
counter packets 5 bytes 342 drop
```

Ponto observado	Origem	Destino
E inicia (G/wan0)	192.0.2.10:porta efêmera	192.0.2.1:18080
Após DNAT (G/dmz0)	192.0.2.10:porta efêmera	10.10.20.20:8080
S responde (G/dmz0)	10.10.20.20:8080	192.0.2.10:porta efêmera
Após tradução de retorno (G/wan0)	192.0.2.1:18080	192.0.2.10:porta efêmera

A tabela da Edição 1 era uma previsão; agora as capturas a confirmam com a mesma porta efêmera e os mesmos números de sequência dos dois lados de G. O conntrack mostra a tupla original (para 192.0.2.1:18080) e a de retorno (de 10.10.20.20:8080). E2 não foi autorizado: o DNAT ainda se aplica, mas o forward descarta. Tradução e autorização são etapas diferentes.

A posição relativa de NAT e filtragem é própria de cada plataforma: no pfSense, o port forward com regra associada é avaliado sobre o destino traduzido [16, 25]. NAT não cria listener: sem serviço na porta interna, nenhuma tradução fará responder.

16 IPv6 e descoberta de MTU

Uma aplicação pode escolher IPv6 quando recebe um registro AAAA; validar só o A e a política IPv4 deixa um caminho sem investigação. ICMPv6 é parte da operação: Neighbor Discovery e mensagens de erro são necessárias, e a RFC 4890 discute filtragem preservando funções essenciais [17]. No IPv6, a descoberta de MTU usa Packet Too Big; se essas mensagens não chegam, parte do tráfego falha de modo aparentemente seletivo [18].

Verificação	Evidência a registrar
DNS A e AAAA	Endereço efetivamente selecionado
Conexão por IPv4 / por IPv6	Resultado e tupla de cada uma, separados
ICMPv6	Tipo, código, origem e pacote citado
Túnel	MTU configurada e pontos de encapsulamento

Em dual stack, documente políticas e testes nas duas famílias. Numa tabela `inet` do nftables, condições `ip saddr` valem só para IPv4: o restante do IPv6 cai na política da cadeia. O laboratório 8 demonstra isso no Windows; a variante Linux não pôde ser executada nesta edição (capítulo 33).

17 TLS, inspeção e visibilidade

Sem terminar ou decifrar a sessão, um intermediário observa parte dos metadados, não o corpo de uma requisição HTTPS. Inspeção TLS exige arquitetura explícita: duas sessões, confiança em certificados, validação do servidor, compatibilidade e capacidade. Pinning e autenticação mútua podem exigir tratamento próprio; no exemplo do App-ID, a aplicação pode ser reavaliada após a decriptação [7].

Situação	O que investigar
HTTPS conecta, aplicação falha	Certificado, autenticação e logs da aplicação
Falha só com inspeção	Confiança, compatibilidade e regra de decriptação
QUIC sobre UDP	Política e capacidade específicas para esse transporte
DoH / DoT	Política de resolução e identificação possível
Exceção de inspeção	Escopo, motivo, responsável e prazo de revisão

Implante com grupo de teste e aplicações conhecidas; valide login, transferência, atualização e encerramento; documente o que ficou fora da inspeção. Evite escrever "todo tráfego foi inspecionado" sem definir critérios e verificar registros.

18 Windows: inventário antes de alterar

Use PowerShell com privilégios administrativos. Os exemplos têm como referência Windows 11 e Windows Server com o módulo NetSecurity; registre edição, build e origem da política. Em domínio, GPO ou MDM pode controlar ou limitar regras locais.

```
Get-NetConnectionProfile
Get-NetFirewallProfile -PolicyStore ActiveStore |
  Select-Object Name, Enabled, DefaultInboundAction, DefaultOutboundAction
Get-NetFirewallRule -PolicyStore ActiveStore | Where-Object Enabled -eq 'True' |
  Select-Object Name, Direction, Action, Profile
```

POWERSHELL NÃO EXECUTADO Leitura.

O perfil da interface precisa corresponder ao escopo da regra. ActiveStore consulta o conjunto efetivo; uma regra no armazenamento local não prova que ela decide o fluxo [19].

```
$r = Get-NetFirewallRule -PolicyStore ActiveStore -Name 'LAB-HTTP-A'
$r | Get-NetFirewallPortFilter
$r | Get-NetFirewallAddressFilter
$r | Get-NetFirewallApplicationFilter
```

Um campo inadequado (endereço remoto, porta local, protocolo, programa) impede a correspondência mesmo com a regra habilitada.

inventario_firewall.ps1 NOVO

Reúne o capítulo num só comando de leitura: perfis, configuração efetiva de cada perfil, **todas as regras habilitadas que tocam a porta** com seus filtros, aviso de bloqueio explícito (que prevalece) e de permissão sem restrição de origem (que invalida o teste negativo), e as últimas linhas do log lidas pelo cabeçalho `#Fields` do próprio arquivo. Listagem no Apêndice E.

```
.\inventario_firewall.ps1 -Porta 8080 -Perfil Private
```

Linha de base: registre acesso local ao serviço, acesso remoto, IP de origem, perfil e horário. Não altere a categoria da rede só para "fazer a regra funcionar"; primeiro entenda por que ela foi classificada assim.

19 Windows: criar, registrar e remover

O exemplo permite TCP/8080 no servidor só para 10.10.10.10, em rede de laboratório com perfil Private e listener ativo. Na interface gráfica, `wf.msc` mostra os mesmos atributos.

```
# MUDANÇA (laboratório)
New-NetFirewallRule -Name 'LAB-HTTP-A' -DisplayName 'LAB HTTP somente cliente A' `
  -Direction Inbound -Action Allow -Profile Private -Protocol TCP -LocalPort 8080 `
  -RemoteAddress 10.10.10.10
```

Para limitar a um programa, use `-Program` com o caminho completo do executável; para um serviço, avalie `-Service` [20].

Log do firewall

O arquivo padrão é `%windir%\system32\logfiles\firewall\pfirewall.log`. A Microsoft recomenda registrar pacotes descartados e conexões permitidas e aumentar o tamanho para pelo menos 20.480 KB; o máximo é 32.767 KB. O serviço `NT SERVICE\mpssvc` precisa de controle total na pasta do log: se você mudar o caminho e o serviço não puder gravar, o log fica vazio sem erro visível [21]. NOVO

```
Get-NetFirewallProfile -Name Private | Select-Object LogAllowed, LogBlocked, LogFileName,
LogMaxSizeKilobytes
Set-NetFirewallProfile -Name Private -LogBlocked True -LogAllowed True      # MUDANÇA
```

O arquivo começa com uma linha `#Fields:` que nomeia as colunas (data, hora, ação, protocolo, endereços e portas, entre outras). Leia pelo cabeçalho, não por posição fixa, como faz `inventario_firewall.ps1`.

```
Remove-NetFirewallRule -Name 'LAB-HTTP-A'        # MUDANÇA: remove só a regra do exercício
```

`lab3_regra.ps1` faz a criação e a remoção e guarda os valores anteriores de log num arquivo, para restaurá-los exatamente. Não use remoção geral de regras para "voltar ao padrão". Se uma regra ampla preexistente já permite a porta, o teste negativo não demonstra a restrição pretendida.

20 Windows: evidência no Filtering Platform NOVO

O Windows Firewall é implementado sobre o Windows Filtering Platform (WFP). Com a auditoria correspondente habilitada, o log Security registra as decisões do WFP com mais detalhe do que o `pfirewall.log`, inclusive **qual filtro bloqueou**.

Evento	Significado	Subcategoria (GUID)
5157	O WFP bloqueou uma conexão [31]	Filtering Platform Connection {0CCE9226-...}
5156	O WFP permitiu uma conexão [33]	Filtering Platform Connection {0CCE9226-...}
5152	O WFP bloqueou um pacote; gerado para cada pacote recebido [32]	Filtering Platform Packet Drop {0CCE9225-...}

GUIDs da especificação MS-GPAC; terminam em `-69AE-11D9-BED3-505054503030` [34].

Campos do 5157

Campo XML	Leitura [31]
Application / ProcessID	Executável (no formato `\device\harddiskvolume#\…`) e PID, correlacionável com o 4688.
Direction	Entrada ou saída (no XML, um código `%%`; o texto aparece na mensagem).
SourceAddress / SourcePort, DestAddress / DestPort, Protocol	A tupla; protocolo 6 = TCP, 17 = UDP, 1 = ICMP.
FilterRTID	ID do filtro WFP que bloqueou.
LayerName / LayerRTID	Camada ALE em que a decisão ocorreu.

Do evento ao filtro

A página do 5157 indica gerar `filters.xml` com `netsh wfp show filters` (sessão elevada) e procurar o `filterId` igual ao FilterRTID; `netsh wfp show state` identifica a camada [31]. O nome do filtro costuma apontar a regra do firewall ou o componente responsável. `evidencias_wfp.ps1` faz os três passos: confere a auditoria por GUID, lista os 5157/5152 da porta e localiza o filtro.

```
.\evidencias_wfp.ps1 -Porta 8080 -Minutos 30 -Pasta C:\Lab\wfp
```

> **VOLUME** Packet Drop gera um evento por pacote, e Connection registra cada conexão permitida ou bloqueada. Habilite só no laboratório ou por janela curta e registre a mudança na política de auditoria (o 4719 da apostila de logs do Windows registra essa alteração).

21 Linux: reconhecer o responsável

Netfilter fornece os mecanismos no kernel; nftables é uma interface para regras; iptables pode operar com backends diferentes. UFW e firewalld são ferramentas de gerenciamento, não mecanismos independentes.

```
uname -r ; nft --version ; iptables --version
sudo nft -a list ruleset
ip -br address ; ip route ; ip -6 route
sudo ss -lntup
```

Quem escreveu as regras? NOVO

Gerenciador	Consulta	Observação
firewalld	`firewall-cmd --state` · `--get-active-zones` · `--list-all --zone=<zona>`	Mudanças em runtime somem no reload se não forem salvas; `--permanent` só vale após reload [38].
UFW	`ufw status verbose` · `ufw show added` · `ufw show raw`	`status` não mostra regras dos arquivos em `/etc/ufw`; `show raw` mostra o estado completo [39].
Contêineres, virtualização, automação	`nft -a list ruleset` e procurar tabelas e cadeias que você não criou	Podem recriar regras no próximo reload.

Uma alteração manual por baixo de um gerenciador pode desaparecer no reload ou conflitar com cadeias existentes. Mude pela ferramenta que é dona da política.

Modelo mental do nftables

Uma tabela agrupa objetos; cadeias contêm regras; base chains têm hook, tipo e prioridade. A família `inet` reúne IPv4 e IPv6, mas `ip saddr` continua só IPv4 [10].

Sequência de mudança: exporte o estado, prepare um arquivo separado, valide com `nft -c -f`, aplique com `nft -f`, confira o carregado, teste e só então trate a persistência. A verificação não prova conectividade nem adequação [12]. Atenção, confirmado no laboratório: **`nft -f` sobre uma tabela que já existe acrescenta regras** ao fim das cadeias em vez de substituí-las; regras novas podem cair depois de um `drop`. Remova a tabela do exercício antes de recarregar.

> **CONSOLE OBRIGATÓRIO** Uma política input drop aplicada remotamente pode cortar a própria administração. Não execute `flush ruleset` em host de uso diário, gateway ou servidor com outras políticas.

22 Linux: regras legíveis e observáveis

```
ct state established,related counter accept
ip saddr 10.10.10.10 tcp dport 8080 counter accept comment "LAB: cliente A"
limit rate 5/second log prefix "LAB_DROP "
counter drop
```

Comentários explicam o uso; contadores mostram correspondências. O `limit` da terceira linha restringe só o **registro**: pacotes acima da taxa não casam com ela e seguem para o `drop` da linha seguinte. Colocar limit e drop na mesma regra mudaria a intenção.

Onde o log aparece

O `log` do nftables vai para o log do kernel (`dmesg` ou `journalctl -k`). Em namespaces de rede e contêineres, o kernel **não registra por padrão**: `net.netfilter.nf_log_all_netns` vale 0 e só o namespace inicial loga, para que contêineres não inundem o log do host [36]. Uma regra de log que "não funciona" num contêiner pode ser isso. A alternativa é enviar para o espaço de usuário com `log group` (nflog), lido por ulogd ou aplicação própria [35]. NOVO

Rastrear a decisão NOVO

O trace mostra regra a regra por onde o pacote passou e qual veredito encerrou. Ele só funciona com uma regra que marque os pacotes (`meta nftrace set 1`), numa cadeia de prioridade anterior às suas; delimite ao fluxo do teste para não gerar volume [35].

```
# Rastreamento: marca so as tentativas de B para TCP/8080 (prioridade antes do filtro)
table ip lab_trace {
  chain trace {
    type filter hook prerouting priority -301;
    ip saddr 10.10.10.11 tcp dport 8080 meta nftrace set 1
  }
}
```

EXECUTADO Em G, durante uma tentativa de B com `lab-router.nft` ativo:

```
$ nft -f lab-trace.nft ; nft monitor trace   (em G, durante uma tentativa de B)
trace id … ip lab_trace trace packet: iif "lan0" ether saddr a2:bf:55:0e:8c:ec ether daddr
b6:67:c1:28:a3:d1 ip saddr 10.10.10.11 ip daddr 10.10.20.
trace id … ip lab_trace trace rule ip saddr 10.10.10.11 tcp dport 8080 meta nftrace set 1 (verdict
continue)
trace id … ip lab_trace trace policy accept
trace id … inet lab_router forward packet: iif "lan0" oif "dmz0" ether saddr a2:bf:55:0e:8c:ec ether
daddr b6:67:c1:28:a3:d1 ip saddr 10.10.10.11 ip
trace id … inet lab_router forward rule limit rate 5/second burst 5 packets log prefix "LAB_FWD_DROP
" (verdict continue)
trace id … inet lab_router forward rule counter packets 3 bytes 180 drop (verdict drop)
```

A leitura é direta: o pacote entrou por `lan0`, foi marcado, chegou ao forward de `lab_router`, passou pela regra de log (veredito continue) e terminou na regra `counter drop` (veredito drop). Nenhuma regra de permissão casou. Ao terminar, remova a tabela de trace.

23 Borda: operar pelo caminho real

O pfSense é o exemplo de firewall de borda; a finalidade é aprender regras, estados e NAT numa interface observável, sem prender os conceitos a uma marca.

01 Mapeie interface física ou virtual, endereço, VLAN, zona, gateway e rede conectada. Um nome como LAN não garante que o adaptador esteja ligado à rede imaginada.

02 Verifique qual interface recebe o primeiro pacote.

03 Liste regras automáticas, floating, grupos e regras da interface.

04 Use aliases com nomes claros e conteúdo revisado.

05 Crie uma regra específica, com descrição e log durante o teste.

06 Salve e aplique; confira o estado efetivo e reproduza uma conexão nova.

No pfSense, regras comuns de interface atuam na entrada e criam estado por padrão ao permitir; por isso a resposta de uma sessão autorizada não exige permissão espelhada [2].

Tela	Pergunta
Firewall → Rules	Qual política foi configurada?
Status → System Logs → Firewall	Qual decisão foi registrada?
Diagnostics → States	Qual fluxo está sendo acompanhado?

Restrinja a administração à rede apropriada e mantenha console. A regra automática contra bloqueio administrativo deve ser compreendida antes de alterada. O roteiro pfSense dos laboratórios 4 e 6 está no anexo.

24 Mudança, backup e reversão

Etapa	Registro exigido
Antes	Sintoma, versão, política atual e teste de referência
Preparação	Regra proposta, dependências e acesso por console
Aplicação	Quem alterou, horário e objetos modificados
Validação	Teste permitido, proibido e funções preservadas
Reversão	Condição de disparo e ação exata
Fechamento	Evidências, responsável e próxima revisão

Windows

```
netsh advfirewall export C:\Lab\antes.wfw
```

A pasta deve existir. Para o exercício, prefira remover a regra criada e restaurar os valores de log anotados (`lab3_regra.ps1 -Remover`). Importar uma política inteira substitui configurações locais e deve ser uma restauração deliberada; políticas de domínio continuam com controle próprio.

Linux

```
sudo nft list ruleset > antes.nft
```

O arquivo é evidência e base de restauração, mas carregá-lo sobre uma política existente acrescenta, não substitui (capítulo 21). Uma tabela exclusiva do laboratório facilita a reversão: basta `nft delete table`.

> **CRITÉRIO DE REVERSÃO** "Se a administração pelo caminho previsto falhar, se um cliente proibido acessar o serviço ou se uma dependência essencial parar, reverter a mudança e preservar os registros." Defina esses testes antes de aplicar, enquanto ainda é possível compará-los com a linha de base. O laboratório 9 executa esse ciclo.

25 Preparar o laboratório isolado

Papel	Endereço	Uso
Cliente A	10.10.10.10/24	Origem permitida
Cliente B	10.10.10.11/24	Origem de teste negativo
Gateway G (LAN / DMZ / WAN)	10.10.10.1 / 10.10.20.1 / 192.0.2.1	Roteamento, política, estados e NAT
Servidor S	10.10.20.20/24	HTTP didático em TCP/8080
Cliente externo E / E2	192.0.2.10 / 192.0.2.11	Teste de DNAT permitido / não autorizado
Roteador R	10.10.20.254/24	Gateway errado do caso 2 NOVO
Servidor Windows W	10.10.10.20/24	Laboratórios 3 e 8 (mesmo switch de A e B)

A e B usam o gateway 10.10.10.1; S usa 10.10.20.1; E e E2, 192.0.2.1. Linux e pfSense são alternativas para G, não dois gateways ativos no mesmo endereço.

Duas formas de montar

VMs (a proposta original): switches virtuais internos sem ponte com a rede física, console e snapshots, ferramentas instaladas antes de isolar. Necessário para Windows e pfSense.

Namespaces de rede (usada nos laboratórios Linux desta edição): numa única VM Linux de laboratório, `topologia.sh` cria uma pilha de rede isolada para cada máquina e liga os segmentos LAN, DMZ e WAN por bridges dentro de um namespace "switch", sem tocar na rede da VM. Cada namespace tem suas próprias interfaces, rotas, tabela nftables e conntrack: o comportamento de filtragem é o mesmo de máquinas separadas.

EXECUTADO `sudo ./topologia.sh criar` e o endereçamento resultante:

```
Kernel: 6.18.44-fc-v185  nftables: nftables v1.0.9 (Old Doc Yak #3)  conntrack: conntrack v1.4.8
(conntrack-tools)
A   eth0@if5 10.10.10.10/24
B   eth0@if6 10.10.10.11/24
G   lan0@if7 10.10.10.1/24  dmz0@if8 10.10.20.1/24  wan0@if9 192.0.2.1/24
S   eth0@if10 10.10.20.20/24
E   eth0@if12 192.0.2.10/24
E2  eth0@if13 192.0.2.11/24
R   eth0@if11 10.10.20.254/24
Aviso: kernel sem IPv6; enderecos IPv6 nao configurados.
```

O kernel do ambiente desta edição foi compilado sem IPv6; por isso a topologia criou só IPv4 e o laboratório 8 permanece NÃO EXECUTADO. Requisitos: iproute2, nftables, conntrack, python3, curl e dumpcap/tshark. `executar_laboratorios.sh` roda todos os laboratórios Linux, liga temporariamente o log de netfilter para namespaces (capítulo 22), restaura o valor anterior e destrói a topologia no fim.

26 Laboratório 1: serviço e linha de base

Objetivo: provar que o serviço funciona antes de atribuir uma falha ao firewall. Sem nenhuma regra em G nem em S.

```
mkdir -p ~/lab-web ; printf 'LAB FIREWALL OK\n' > ~/lab-web/index.html
python3 -m http.server 8080 --bind 10.10.20.20 --directory ~/lab-web
ss -lnt | grep ':8080'
curl --noproxy '*' --max-time 5 http://10.10.20.20:8080/
```

EXECUTADO

```
$ ss -lnt   (em S)
State  Recv-Q Send-Q Local Address:Port Peer Address:PortProcess
LISTEN 0      5        10.10.20.20:8080      0.0.0.0:*
$ curl local em S e a partir de A e B, sem nenhuma regra
S    http://10.10.20.20:8080/           -> HTTP 200  tempo 0.006007s  curl_exit=0
A    http://10.10.20.20:8080/           -> HTTP 200  tempo 0.001058s  curl_exit=0
B    http://10.10.20.20:8080/           -> HTTP 200  tempo 0.001035s  curl_exit=0
```

Listener ativo, resposta local e resposta remota de A e B: linha de base válida. O servidor HTTP do Python é didático, não recomendação para produção [24]. Falha local aponta primeiro para listener, bind ou aplicação; falha remota com sucesso local exige olhar caminho e políticas. Se a linha de base falhar, não avance como se tivesse sido validada.

27 Laboratório 2: política de host Linux

Objetivo: permitir só A no serviço de S, com a política no próprio S. O gateway continua permitindo A e B.

```
# Laboratorio 2: politica de host em S (somente A acessa TCP/8080)
table inet lab_host {
  chain input {
    type filter hook input priority 0; policy drop;
    iifname "lo" accept
    ct state invalid drop
    ct state established,related accept
    meta l4proto { icmp, ipv6-icmp } accept
    ip saddr 10.10.10.10 tcp dport 8080 counter accept comment "LAB: cliente A"
    limit rate 5/second log prefix "LAB_HOST_DROP "
    counter drop
  }
  chain output {
    type filter hook output priority 0; policy accept;
  }
}
```

EXECUTADO Em S: `nft -c -f`, carga, testes, contadores, log do kernel e captura em S durante a tentativa de B:

```
$ nft -c -f lab-host.nft && nft -f lab-host.nft   (em S)
A    http://10.10.20.20:8080/           -> HTTP 200  tempo 0.001747s  curl_exit=0
B    http://10.10.20.20:8080/           -> HTTP 000  tempo 3.002187s  curl_exit=28

$ nft list table inet lab_host   (contadores)
                type filter hook input priority filter; policy drop;
                ip saddr 10.10.10.10 tcp dport 8080 counter packets 1 bytes 60 accept comment "LAB:
cliente A"
                counter packets 3 bytes 180 drop
                type filter hook output priority filter; policy accept;

$ dmesg | grep LAB_HOST_DROP
LAB_HOST_DROP IN=eth0 OUT= MAC=7a:45:51:60:5b:8d:be:2a:15:02:35:fa:08:00 SRC=10.10.10.11
DST=10.10.20.20 LEN=60 TOS=0x00 PREC=0x00 TTL=63 ID=15067 DF  […]
LAB_HOST_DROP IN=eth0 OUT= MAC=7a:45:51:60:5b:8d:be:2a:15:02:35:fa:08:00 SRC=10.10.10.11
DST=10.10.20.20 LEN=60 TOS=0x00 PREC=0x00 TTL=63 ID=15068 DF  […]

$ tshark -r 02_S_eth0_B_drop.pcapng   (captura em S durante as tentativas)
14 2026-10-30 15:42:25.096410023  10.10.10.11 → 10.10.20.20  TCP 74 41466 → 8080 [SYN] Seq=2355376465
Win=64240 Le
15 2026-10-30 15:42:26.118038422  10.10.10.11 → 10.10.20.20  TCP 74 [TCP Retransmission] 41466 → 8080
[SYN] Seq=23
16 2026-10-30 15:42:27.142051089  10.10.10.11 → 10.10.20.20  TCP 74 [TCP Retransmission] 41466 → 8080
[SYN] Seq=23

$ nft delete table inet lab_host  -> nova tentativa
B    http://10.10.20.20:8080/           -> HTTP 200  tempo 0.001047s  curl_exit=0
```

Leitura das evidências

- O contador da regra de A registrou 1 pacote: o SYN que abriu a conexão. Os demais pacotes da sessão casaram com `established,related`, que não tem contador.
- O drop contou 3 pacotes de B: o SYN e duas retransmissões (1 s e 2 s depois), as mesmas três linhas da captura. Na captura não há SYN/ACK: S recebeu e não respondeu.

- O log do kernel tem uma linha `LAB_HOST_DROP` por pacote descartado (o script mostra só as duas últimas). O prefixo único permite achar as linhas, e os campos IN, SRC, DST e TTL confirmam que o pacote veio de B pela interface de S. Use o log para localizar e o contador e a captura para quantificar.
- Depois de `nft delete table inet lab_host`, B volta a receber 200: reversão confirmada.

Se B funcionar com a política carregada, procure conexão antiga, outra origem, outro destino ou outra tabela ativa.

28 Laboratório 3: firewall do Windows

NÃO EXECUTADO Use A, B e W (10.10.10.20) no mesmo switch isolado, com W em perfil Private. Em W, crie `C:\Lab\web` com um `index.html` fictício e, com Python instalado:

```
New-Item -ItemType Directory C:\Lab\web -Force
Set-Content C:\Lab\web\index.html 'LAB FIREWALL OK'
py -m http.server 8080 --bind 10.10.10.20 --directory C:\Lab\web
```

CORREÇÃO A Edição 1 trazia `C:Labweb`, sem as barras. No Windows, `C:Labweb` é um caminho **relativo** ao diretório atual da unidade C, então o servidor podia publicar a pasta errada. O caminho agora é `C:\Lab\web` nos laboratórios 3 e 8.

01 Em outra janela administrativa: `Get-NetTCPConnection -LocalPort 8080 -State Listen` e `.\inventario_firewall.ps1 -Porta 8080`. A entrada deve bloquear por padrão e não pode haver outra regra que permita a porta ou o Python para B. Não aceite a liberação ampla do Python pelo aviso do Windows sem revisá-la.

02 `.\lab3_regra.ps1 -Criar` (LAB-HTTP-A e log ligado, valores anteriores guardados).

03 Nos clientes: `Test-NetConnection 10.10.10.20 -Port 8080` e `curl.exe --noproxy "*" --max-time 5 http://10.10.10.20:8080/`. Esperado: A conecta; B falha no TCP. Ping bem-sucedido não prova acesso à porta.

04 Evidência: `.\inventario_firewall.ps1 -Porta 8080` (regras e log) e, com a auditoria do WFP ligada, `.\evidencias_wfp.ps1 -Porta 8080`.

05 Limpeza: `.\lab3_regra.ps1 -Remover` e encerre o Python. Se o resultado divergir, confira regras amplas, bloqueios explícitos e política central, em vez de desabilitar o firewall.

29 Laboratório 4: acompanhar uma sessão

Objetivo: separar criação de estado, continuidade da conexão e nova conexão. A Edição 1 propunha o experimento no pfSense (roteiro no anexo, NÃO EXECUTADO); esta edição o executou no gateway Linux, com `lab-router.nft` ativo e uma conexão de A aberta e ociosa (`cliente_persistente.py`).

EXECUTADO

```
$ A abre uma conexao TCP e fica ociosa (python cliente.py 6)
$ conntrack -L -p tcp --dport 8080   (em G)
tcp      6 107 TIME_WAIT src=10.10.10.10 dst=10.10.20.20 sport=38542 dport=8080 src=10.10.20.20
dst=10.10.10.10 sport=8080 dport=38542 [ASSURED]
tcp      6 431999 ESTABLISHED src=10.10.10.10 dst=10.10.20.20 sport=38976 dport=8080 src=10.10.20.20
dst=10.10.10.10 sport=8080 dport=38976 [ASSURED]
$ nft delete rule inet lab_router forward handle 4   (remove a permissao de A)
$ nova conexao de A
A    http://10.10.20.20:8080/           -> HTTP 000  tempo 3.003049s  curl_exit=28
$ a conexao antiga envia o pedido depois da remocao
conectado de ('10.10.10.10', 38976)
resposta: HTTP/1.0 200 OK

$ variante: remover tambem o estado do fluxo (conntrack -D) antes do pedido
$ conntrack -D -p tcp -s 10.10.10.10 --dport 8080
  conntrack v1.4.8 (conntrack-tools): 3 flow entries have been deleted.
conectado de ('10.10.10.10', 35298)
erro: TimeoutError
```

Leitura das evidências

- O conntrack mostra a sessão ociosa de A como ESTABLISHED, com as duas tuplas e um tempo restante de 431.999 s (cerca de 5 dias, o padrão do kernel para TCP estabelecido). A linha TIME_WAIT é de um teste anterior.
- Removida a regra de A, a **conexão nova falha** (3 s, timeout), mas a **conexão antiga envia o pedido e recebe 200**: seus pacotes casam com `established,related accept`, que vem antes e não depende da regra removida.
- Na variante, remover também o estado com `conntrack -D` fez a conexão antiga falhar: sem a entrada, o próximo pacote deixa de ser "established" e cai no drop.

> **CONSEQUÊNCIA OPERACIONAL** Remover uma permissão não encerra sessões já estabelecidas neste modelo. Para cortar um acesso de verdade, a mudança precisa incluir a remoção dos estados do fluxo, de forma delimitada. Outros produtos podem reavaliar estados de modo diferente: teste e registre a versão.

30 Laboratório 5: encaminhamento Linux

Objetivo: filtrar no gateway o tráfego que o atravessa. Registre o valor anterior de `net.ipv4.ip_forward` e habilite-o; a mudança por sysctl é temporária.

```
# Laboratorio 5: encaminhamento em G (lan0 = LAN, dmz0 = DMZ; no manual original, ens18/ens19)
table inet lab_router {
  chain forward {
    type filter hook forward priority 0; policy drop;
    ct state invalid drop
    ct state established,related counter accept
    iifname "lan0" oifname "dmz0" \
      ip saddr 10.10.10.10 ip daddr 10.10.20.20 \
      tcp dport 8080 counter accept comment "LAB: A para S:8080"
    limit rate 5/second log prefix "LAB_FWD_DROP "
    counter drop
  }
}
```

A tabela não tem input: não filtra serviços destinados ao próprio gateway e não deve ser exposta como política completa de borda. O resultado executado está no capítulo 11: A recebe 200, B é descartado, o contador do drop registra os 3 SYN de B, o log mostra `LAB_FWD_DROP IN=lan0 OUT=dmz0`, e as capturas mostram os SYN de B só em `lan0`.

Diagnóstico pela captura em G	Próximo passo
SYN entra e não sai	Contador do drop e rota
SYN sai e não retorna	S e o gateway de S (capítulo 36)
Resposta entra e não sai	Retorno e estado

Limpeza: `nft delete table inet lab_router` e restaurar `ip_forward`.

31 Laboratório 6: publicar na DMZ

Objetivo: publicar S:8080 como 192.0.2.1:18080 só para E. A Edição 1 usava o pfSense (roteiro no anexo, NÃO EXECUTADO): Firewall → NAT → Port Forward com WAN / TCP / origem 192.0.2.10 / destino WAN address porta 18080 / redirect 10.10.20.20:8080 / regra associada / descrição LAB-DNAT-HTTP [25]. No ambiente isolado, verifique se o bloqueio de bogons e redes reservadas afeta 192.0.2.0/24 e ajuste só a WAN de teste.

Variante Linux executada

```
# Laboratorio 6 (variante Linux): publicar S:8080 em 192.0.2.1:18080 so para E
table ip lab_nat {
  chain prerouting {
    type nat hook prerouting priority dstnat;
    iifname "wan0" tcp dport 18080 counter dnat to 10.10.20.20:8080 comment "LAB-DNAT-HTTP"
  }
}
table inet lab_pub {
  chain forward {
    type filter hook forward priority 0; policy drop;
    ct state invalid drop
    ct state established,related counter accept
    iifname "wan0" oifname "dmz0" ip saddr 192.0.2.10 ip daddr 10.10.20.20 \
      tcp dport 8080 ct status dnat counter accept comment "LAB: E publicado"
    limit rate 5/second log prefix "LAB_PUB_DROP "
    counter drop
  }
}
```

Duas tabelas, duas funções: `lab_nat` traduz e `lab_pub` autoriza. A condição `ct status dnat` exige que o fluxo tenha passado pela tradução, para que S não seja alcançável diretamente pela WAN com o endereço interno. O resultado executado está no capítulo 15: E recebe 200, E2 é descartado, e as capturas e o conntrack mostram as tuplas antes e depois da tradução.

Limpeza: remover as duas tabelas e conferir que não restaram estados do teste. Testar pelo endereço externo a partir da LAN é outro cenário (reflexão ou hairpin) e não substitui o teste externo.

32 Laboratório 7: provar a hipótese

Sincronize os relógios, faça uma reprodução curta com um único cliente e mantenha capturas separadas por ponto. A Edição 1 trazia uma linha do tempo fictícia; esta é real.

EXECUTADO Servidor parado; A tenta S:8080; captura simultânea em A, G/lan0, G/dmz0 e S, ordenada em UTC (flags: S = SYN, A = ACK, R = RST):

```
$ servidor parado; A tenta S:8080 com 4 pontos de captura
A    http://10.10.20.20:8080/           -> HTTP 000  tempo 0.000136s  curl_exit=7
15:43:25.922187 UTC  A       10.10.10.10 -> 10.10.20.20  ··········S·
15:43:25.922202 UTC  G/lan0  10.10.10.10 -> 10.10.20.20  ··········S·
15:43:25.922209 UTC  G/dmz0  10.10.10.10 -> 10.10.20.20  ··········S·
15:43:25.922215 UTC  S       10.10.10.10 -> 10.10.20.20  ··········S·
15:43:25.922224 UTC  S       10.10.20.20 -> 10.10.10.10  ·······A·R··
15:43:25.922227 UTC  G/dmz0  10.10.20.20 -> 10.10.10.10  ·······A·R··
15:43:25.922229 UTC  G/lan0  10.10.20.20 -> 10.10.10.10  ·······A·R··
15:43:25.922230 UTC  A       10.10.20.20 -> 10.10.10.10  ·······A·R··
```

O SYN de A aparece nos quatro pontos, em ordem, em poucas dezenas de microssegundos; o RST/ACK nasce em S e volta pelo mesmo caminho. Conclusão no formato pedido: "O SYN atravessou G e S respondeu RST; não havia listener 8080 no horário. Não há evidência de drop em G para essa tentativa." Note que a mesma recusa, vista só no cliente, seria igual ao reject do capítulo 3: foi a captura em S que localizou a origem.

Filtros de exibição úteis (validados no Wireshark 4.2 na apostila de tráfego): `tcp.port == 8080`, `tcp.flags.syn == 1 && tcp.flags.ack == 0`, `tcp.flags.reset == 1`, `icmp || icmpv6` [26].

33 Laboratório 8: comparar IPv4 e IPv6

NÃO EXECUTADO O kernel do ambiente desta edição não tem suporte a IPv6, então nem a variante Linux pôde rodar. Use o switch isolado do laboratório 3, com A = `2001:db8:10::10/64` e W = `2001:db8:10::20/64` (prefixo de documentação, não anunciar na rede real).

```
py -m http.server 8081 --bind 2001:db8:10::20 --directory C:\Lab\web
New-NetFirewallRule -Name 'LAB-V6-A' -DisplayName 'LAB IPv6 cliente A' -Direction Inbound `
  -Action Allow -Profile Private -Protocol TCP -LocalPort 8081 -RemoteAddress 2001:db8:10::10

curl.exe -4 --noproxy "*" --max-time 5 http://10.10.10.20:8080/
curl.exe -6 --noproxy "*" --max-time 5 "http://[2001:db8:10::20]:8081/"
```

Esperado: cada teste funciona com o listener e a regra correspondente. Remova só LAB-V6-A e repita: a permissão IPv4 não autoriza a origem IPv6. Capture `ipv6` e `icmpv6` para ver descoberta de vizinhos e sessão TCP; não bloqueie ICMPv6 para fabricar a falha. O laboratório não mede PMTUD, pois fica num só segmento.

No Linux, o equivalente seria acrescentar endereços IPv6 à topologia (o `topologia.sh` já os cria quando o kernel tem IPv6) e repetir o laboratório 5: com `lab-router.nft`, que só tem regra `ip saddr`, o acesso IPv6 de A cai no `policy drop` até que se acrescente a regra `ip6` equivalente.

34 Laboratório 9: ensaiar a reversão

Cenário: o serviço 8080 está disponível para A e será substituído por 8082, preservando o acesso só de A; a porta antiga deixa de aceitar conexões novas. Depois, simula-se a reversão.

EXECUTADO No gateway Linux, com listeners 8080 e 8082 em S:

```
$ 1. linha de base (lab-router.nft, 8080)
A    http://10.10.20.20:8080/           -> HTTP 200  tempo 0.003985s  curl_exit=0
B    http://10.10.20.20:8080/           -> HTTP 000  tempo 3.002397s  curl_exit=28
$ 2-4. troca pela regra 8082 (arquivo novo validado com nft -c)
A    http://10.10.20.20:8082/           -> HTTP 200  tempo 0.004275s  curl_exit=0
B    http://10.10.20.20:8082/           -> HTTP 000  tempo 3.003058s  curl_exit=28
A    http://10.10.20.20:8080/           -> HTTP 000  tempo 3.002982s  curl_exit=28
$ 5-6. reversao: volta lab-router.nft e repete os testes
A    http://10.10.20.20:8080/           -> HTTP 200  tempo 0.001303s  curl_exit=0
B    http://10.10.20.20:8080/           -> HTTP 000  tempo 3.002951s  curl_exit=28
A    http://10.10.20.20:8082/           -> HTTP 000  tempo 3.002966s  curl_exit=28
```

Passo	Ação	Critério observado
1	Política atual (8080)	A acessa 8080; B não
2–4	Arquivo novo validado; troca da tabela	A acessa 8082; B não; 8080 recusa conexões novas
5–6	Reversão para o arquivo original	Volta à linha de base: A em 8080; 8082 fechado

A troca foi feita removendo a tabela e carregando o arquivo novo, nunca recarregando por cima (capítulo 21). Registre cada comando realmente aplicado e, se algo falhar, o que observou, não o que esperava. Sessões abertas antes da troca continuam pelo estado (capítulo 29): decida explicitamente se elas devem ser encerradas.

35 Caso 1: a regra existe, mas falha

SINTÉTICO A tenta acessar W:8080. O operador mostra uma regra allow habilitada e conclui que "não pode ser firewall".

Evidência	Observação
Teste local	HTTP responde em W
Captura em W	SYN de A chega, sem SYN/ACK
Perfil da interface	Public
Perfil da regra LAB-HTTP-A	Private
Log de descarte	Corresponde a A, W e porta 8080 no horário
Evento 5157 NOVO	Conexão bloqueada, A → W:8080; o FilterRTID aponta, no `filters.xml`, um filtro de bloqueio padrão do perfil, não LAB-HTTP-A

Raciocínio. O serviço local responde e o pacote chega à interface. A regra exibida não se aplica ao perfil ativo. O log e o 5157 sustentam o bloqueio no host, e o FilterRTID identifica qual filtro decidiu. A existência da regra no console não demonstrava sua aplicabilidade.

Correção. Verifique primeiro por que a interface está Public. Se a classificação estiver correta, avalie uma permissão restrita aplicável ao perfil necessário; se o ambiente deveria estar em domínio, investigue a detecção do domínio. Não mude o perfil indiscriminadamente para reduzir proteção.

Validação. Repita A e B, confirme a regra efetiva (`inventario_firewall.ps1`) e registre a nova decisão. A acessa; B segue bloqueado.

Variante. Se há bloqueio explícito conflitante, criar outra permissão não resolve, por mais específica que seja [11]. Investigue a política que produz o block e seu propósito.

36 Caso 2: DNAT correto, retorno errado

E acessa 192.0.2.1:18080, G traduz para S:8080 e o cliente termina em timeout. Na Edição 1 o caso era sintético; aqui foi reproduzido trocando o gateway padrão de S para o roteador R (10.10.20.254), que não conhece a tradução.

EXECUTADO Captura em S e em G/dmz0 durante a tentativa de E; depois, a rota corrigida:

```
$ ip route replace default via 10.10.20.254   (em S: gateway errado R)
E    http://192.0.2.1:18080/            -> HTTP 000  tempo 3.002565s  curl_exit=28
Em S:     SYN de E recebidos 3, SYN/ACK enviados 5
Em G/dmz0: SYN/ACK para E vistos 0
Destino L2 do SYN/ACK em S (MAC de R x MAC de G):
  MAC de G dmz0: be:2a:15:02:35:fa   MAC de R: 3e:df:92:02:36:c8
  SYN/ACK para 5 3e:df:92:02:36:c8

$ ip route replace default via 10.10.20.1   (corrigido)
E    http://192.0.2.1:18080/            -> HTTP 200  tempo 0.001122s  curl_exit=0
```

Raciocínio. S recebeu os SYN traduzidos e respondeu com SYN/ACK, mas nenhum chegou a G/dmz0. O endereço MAC de destino dos SYN/ACK é o de R, não o de G: a resposta seguiu o gateway errado, que não tem o mapeamento. A falha está no retorno; uma captura só na WAN mostraria apenas "SYN sem resposta". Os números de SYN e SYN/ACK diferem por causa das retransmissões dos dois lados.

Correção e teste. Corrija a rota conforme a arquitetura (aqui, de volta para 10.10.20.1) e repita com uma conexão nova: HTTP 200. Não adicione SNAT amplo só para esconder o problema; avalie a perda da identidade de origem nos logs. A documentação de diagnóstico de port forward da Netgate pede justamente verificar chegada, tradução e caminho de resposta, incluindo o gateway do servidor [27].

Variante sem NAT. Mesmo sem tradução, um firewall stateful que vê só parte da conversa pode considerar pacotes inválidos. A solução é simetria, sincronização ou arquitetura suportada; "desligar a inspeção stateful" não é diagnóstico.

37 Caso 3: por IP funciona, por nome não

SINTÉTICO curl para o IPv4 do servidor funciona; a URL com nome falha. Suspeita-se de bloqueio de HTTPS.

01 Consulte A e AAAA no resolvedor realmente usado pelo cliente.

02 Registre o endereço escolhido pela tentativa que falha.

03 Verifique proxy, VPN e rota para esse endereço.

04 Compare TCP, TLS e HTTP separadamente.

05 Consulte o log da aplicação ou do WAF para a requisição.

Descoberta possível	Hipótese a testar
A aponta para IP antigo	Cache ou zona incorreta
AAAA existe e IPv6 falha	Caminho ou política IPv6 incompleta
IP conecta, TLS com nome falha	Certificado, SNI ou inspeção
HTTP 403 só com nome	Virtual host, WAF ou autorização
Cliente usa proxy	A conexão observada pode terminar no proxy

Em HTTPS, trocar o nome pelo IP altera validação de certificado e virtual host; um teste TCP por endereço não reproduz a transação. Mantenha o nome quando a pergunta for sobre TLS ou HTTP e controle a resolução no cliente de teste. **Resolução proposta:** o AAAA apontava para um endereço sem política equivalente e o cliente usava IPv6 na tentativa que falhava. Corrigidos caminho e política IPv6, as duas famílias foram testadas e os registros confirmados.

38 Caso 4: domínio e e-mail

Active Directory: liberar uma porta não basta

Um recurso do domínio pode depender de DNS, Kerberos, LDAP, SMB, horário e RPC, conforme a operação; nem todas as portas de uma tabela são necessárias em todos os ambientes [28, 29]. O RPC usa o endpoint mapper em TCP 135 e depois uma porta dinâmica; desde o Windows Server 2008, a faixa padrão é 49152–65535 [29].

Etapa	Evidência desejada
Localizar o serviço	Consulta DNS e resposta do resolvedor usado
Autenticar	Log do cliente ou do DC com horário correlacionável
Acessar o recurso	Fluxo específico e resultado da aplicação
Operação RPC	Endpoint inicial e porta negociada

SINTÉTICO Login funciona, mas uma operação administrativa falha. A equipe havia liberado TCP/135 e presumido que todo RPC estava coberto; a captura revelou a conexão adicional à porta dinâmica. A correção foi delimitada aos hosts e à operação, com teste negativo fora do escopo.

E-mail: transporte e aceitação são etapas diferentes

Um gateway de e-mail alcançável por TCP pode rejeitar a mensagem após o diálogo SMTP. Uma resposta 5xx é decisão de quem fala SMTP, não drop de pacote. Num relay negado, mudar o firewall não resolve: correlacione a conversa SMTP com o log do MTA e o identificador da mensagem.

Resultado: antes de liberar mais portas, determine em qual etapa a transação parou: transporte, autenticação, autorização ou política de conteúdo. Isso conecta esta apostila às de DNS, entrega de e-mails, logs do Windows e tráfego com Wireshark.

39 Projetar zonas e dependências

Uma zona representa um conjunto com requisitos de comunicação e confiança semelhantes. Uma VLAN separa segmentos, mas não substitui a política entre eles nem o controle no mesmo segmento.

Origem	Destino	Intenção	Decisão proposta
Usuários	Portal na DMZ	Consulta HTTPS	Permitir o serviço necessário
Usuários	Gestão do firewall	Administração	Negar
Administração	Hosts geridos	Gestão autorizada	Permitir destinos e serviços definidos
DMZ	Banco interno	Dependência da aplicação	Permitir só servidor e porta previstos
Visitantes	Redes internas	Sem necessidade	Negar
Servidores	DNS interno	Resolução	Permitir resolvedores definidos

Proposta didática, não configuração universal. Para cada linha, acrescente endereços, família IP, protocolo, portas, responsável e teste de comprovação.

Separar planos. A administração do firewall, os serviços que ele oferece e o tráfego que o atravessa exigem políticas distintas; o capítulo 11 mostrou na prática que input e forward são caminhos diferentes. **Microsegmentação** no host, hipervisor ou controle distribuído depende de inventário e identidade confiáveis; uma regra ampla distribuída a muitos hosts continua ampla. **Revisão de dependências:** comece por uma aplicação (quem inicia, backends, resolução de nomes, autenticação, atualização, registro) e só depois agrupe regras.

> **CRITÉRIO DE QUALIDADE** Cada permissão deve ter uma necessidade verificável e um responsável capaz de confirmar se ainda é necessária.

40 Alta disponibilidade e capacidade

Dois equipamentos não garantem continuidade se dependem do mesmo enlace, switch, alimentação ou configuração incorreta. No pfSense, CARP fornece redundância de endereço, XMLRPC sincroniza configuração e pfsync sincroniza estados: funções diferentes, com verificação própria [30].

Ensaio de failover: mantenha uma conexão longa e abra conexões novas durante a troca controlada; registre perda, tempo de recuperação e necessidade de reconexão; verifique NAT, rotas, VPN e administração. Um ping que volta a responder não prova preservação das sessões. O capítulo 29 mostrou o que acontece com uma sessão quando o estado some: o equivalente num failover sem sincronização de estados.

Métrica	Pergunta prática
Throughput com inspeção	O desempenho considera os recursos ativos?
Novas conexões por segundo	O pico de abertura cabe na plataforma?
Sessões simultâneas	Há capacidade e margem para o volume real?
CPU e memória	Há saturação sustentada ou picos correlacionados?
Drops de interface	O problema ocorre antes da decisão de política?
Volume de logs	Há perda, atraso ou retenção insuficiente?

Um número de throughput sem IPS e TLS não representa a capacidade com esses recursos. Após atualização de firmware, assinaturas ou classificação de aplicações, valide comportamento: "configuração igual" não garante decisão igual.

A Exercícios

Responda antes de consultar o gabarito, com conclusão e evidência necessária. Sugestão: 1 ponto por item, metade pela conclusão e metade pela justificativa.

01 Uma ACL stateless permite cliente:53024 → servidor:443. Uma regra de retorno com destino 443 basta? Escreva a tupla de retorno.

02 Um firewall stateful mantém entradas UDP. Isso contradiz o fato de UDP não ter handshake?

03 No Windows há allow para um host e block para a sub-rede que o inclui, mesmo serviço e perfil. A especificidade do allow resolve?

04 Um pacote para um servidor atrás de um gateway Linux é liberado em input. Qual caminho deveria ser examinado?

05 O SYN entra no firewall e sai para o servidor; o servidor responde RST. Que duas verificações localizam a causa?

06 Uma regra de DNAT traduz a porta externa. Isso prova autorização e serviço na porta interna?

07 Após remover uma regra, uma sessão antiga continua e uma nova falha. Que mecanismo explica?

08 IPv4 funciona, mas a tentativa pelo nome usa AAAA. O que testar antes de culpar a aplicação?

09 O WAF registra uma regra em modo de contagem. Um 403 prova que ela bloqueou?

10 O log do firewall não registra a tentativa. Isso prova que não houve descarte? Cite duas limitações.

11 NOVO O cliente recebe "connection refused" imediato. Isso prova que não há serviço na porta?

12 NOVO A regra de log de um firewall em contêiner não registra nada no `dmesg`, embora o contador do drop suba. Hipótese?

13 NOVO Você recarregou com `nft -f` um arquivo corrigido e a regra nova não funciona, mas aparece no ruleset. O que procurar?

Exercício integrador

Desenhe A, gateway, servidor e os pontos de captura. Escreva uma política que permita A:origem efêmera → S:8080 e negue B. Descreva os testes que distinguem serviço parado, filtro no gateway e filtro no servidor. Compare com as saídas dos capítulos 3, 11, 27 e 32.

B Gabarito comentado

1. Não. O retorno é servidor:443 → cliente:53024; uma ACL sem estado não herda a permissão de resposta.

2. Não. O firewall acompanha endereços, portas, direções e tempo; o estado do mecanismo não exige handshake.

3. Não. O bloqueio explícito prevalece; a especificidade só decide entre permissões. Verifique a política efetiva e os filtros.

4. Forward. Input trata entrega ao próprio gateway (capítulo 11, contador 0 na regra de input). Confirme encaminhamento habilitado, interfaces e rotas.

5. Listener e origem do RST: sockets e logs no servidor e captura no ponto de emissão (capítulo 32). Um RST no cliente não prova quem o gerou.

6. Não. Tradução, autorização e aplicação são etapas distintas (capítulo 15: E2 traduzido e descartado).

7. Estado preexistente (capítulo 29). Teste conexão nova e confira o comportamento do produto.

8. O caminho IPv6 completo: endereço escolhido, listener, política IPv6, rotas e ICMPv6.

9. Não. Contagem observa; o bloqueio pode ser de outra regra ou da aplicação. Correlacione ação terminante e identificador da requisição.

10. Não. O log pode estar desligado, limitado, perdido ou em outro namespace; a consulta pode usar horário, interface ou filtro errados; o pacote pode não ter chegado. Combine registros, contadores e captura.

11. Não. Um reject com RST produz exatamente o mesmo resultado no cliente (capítulo 3). Confira o listener no servidor e capture no servidor.

12. `net.netfilter.nf_log_all_netns` vale 0: só o namespace inicial registra no log do kernel (capítulo 22). Use `log group` ou ajuste no host de laboratório.

13. Regras duplicadas: `nft -f` sobre tabela existente acrescenta ao fim da cadeia, e a regra nova pode estar depois de um drop. Remova a tabela e recarregue (capítulo 21).

Integrador. A resposta completa descreve a 5-tupla, o ponto de decisão e testes positivos e negativos. Serviço parado exige evidência do host (listener e captura em S com RST); filtro no gateway exige observação nas duas interfaces do gateway (SYN entra e não sai); filtro no host exige política, log e captura locais (SYN chega e não há resposta). Declare os limites da coleta.

C Checklist e registro de mudança

Antes de alterar

☐	Sintoma, horário e tentativa reproduzível definidos
☐	Origem, destino, protocolo, portas e família IP registrados
☐	Caminho, NAT, VPN, proxy e retorno desenhados
☐	Serviço em escuta e política efetiva conferidos; gerenciador identificado
☐	Backup, console e procedimento de reversão disponíveis

Depois de alterar

☐	Cliente permitido executa a operação completa
☐	Cliente proibido continua sem acesso
☐	Administração, DNS e dependências preservados
☐	Sessões novas e antigas avaliadas; estados removidos se o objetivo for cortar acesso
☐	Logs, contadores e capturas sustentam a conclusão
☐	Persistência revisada; regras temporárias e logging adicional removidos

Registro

Campo	Preenchimento
Identificador / responsável	
Necessidade / aplicação	
Origem / zona / família	
Destino antes e depois do NAT	
Protocolo / portas / direção	
Regra / local de aplicação	
Validade / data de revisão	
Testes positivos e negativos	
Condição e ação de reversão	
Evidências / conclusão / limites	

Exemplo de conclusão: "A nova regra permite somente a origem prevista em sessões novas; o cliente B permanece bloqueado. Foram verificados HTTP, logs e captura no intervalo registrado. Não foi testado failover nesta mudança."

D Comandos e glossário

Objetivo	Comando (leitura, salvo indicação)	Cap.
Perfil e política Windows	`Get-NetConnectionProfile` · `Get-NetFirewallRule -PolicyStore ActiveStore`	18
Inventário de uma porta	`.\inventario_firewall.ps1 -Porta 8080`	18
Bloqueio no WFP	Eventos 5157/5152 · `netsh wfp show filters` · `.\evidencias_wfp.ps1`	20
Listener Windows / Linux	`Get-NetTCPConnection -State Listen` · `ss -lntup`	18, 21
Regras Linux	`nft -a list ruleset` · `firewall-cmd --list-all` · `ufw status verbose`	21
Validar arquivo	`nft -c -f arquivo.nft`	21
Estados Linux	`conntrack -L` · `conntrack -E` · (MUDANÇA) `conntrack -D` com filtro	13
Rastrear decisão	regra `meta nftrace set 1` + `nft monitor trace`	22
Log do kernel	`dmesg` · `journalctl -k`	22
Testes	`curl --noproxy '*' --max-time 5 URL` · `Test-NetConnection HOST -Port P`	26

Termo	Significado
ACL	Lista de condições e ações de controle de acesso
Estado / conntrack	Contexto de fluxo mantido pelo mecanismo / acompanhamento de conexões do Netfilter
Hook / base chain	Ponto do caminho em que regras são avaliadas / cadeia ligada a um hook
DNAT / SNAT	Tradução de destino / de origem
WFP	Windows Filtering Platform, base do Windows Firewall
Namespace de rede	Pilha de rede isolada dentro de um kernel Linux
Egress	Tráfego de saída em relação ao ponto definido
Rollback	Reversão planejada de uma alteração

E Scripts do laboratório

E.1 Linux (EXECUTADO)

topologia.sh

```
#!/usr/bin/env bash
# Cria a topologia do manual em namespaces de rede isolados (Linux, root).
#   LAN 10.10.10.0/24: A .10, B .11, G .1      DMZ 10.10.20.0/24: S .20, G .1, R .254
#   WAN 192.0.2.0/24:  E .10, E2 .11, G .1     IPv6: LAN 2001:db8:10::/64, DMZ 2001:db8:20::/64
# Os segmentos sao bridges dentro do namespace SW: nada toca a rede da maquina.
# Uso: sudo ./topologia.sh criar | destruir
set -e
NSS="A B G S E E2 R SW"
destruir() { for n in $NSS; do ip netns del "$n" 2>/dev/null || true; done; }
criar() {
  destruir
  for n in $NSS; do ip netns add "$n"; ip -n "$n" link set lo up; done
  for br in lan dmz wan; do ip -n SW link add "br-$br" type bridge; ip -n SW link set "br-$br" up; done
  liga() {  # liga <ns> <if> <bridge> <ipv4/mask>
    ip link add "$2" netns "$1" type veth peer name "sw-$1-$2" netns SW
    ip -n SW link set "sw-$1-$2" master "br-$3"; ip -n SW link set "sw-$1-$2" up
    ip -n "$1" addr add "$4" dev "$2"; ip -n "$1" link set "$2" up
  }
  liga A eth0 lan 10.10.10.10/24;  liga B eth0 lan 10.10.10.11/24
  liga G lan0 lan 10.10.10.1/24;   liga G dmz0 dmz 10.10.20.1/24;  liga G wan0 wan 192.0.2.1/24
  liga S eth0 dmz 10.10.20.20/24;  liga R eth0 dmz 10.10.20.254/24
  liga E eth0 wan 192.0.2.10/24;   liga E2 eth0 wan 192.0.2.11/24
  for n in A B; do ip -n "$n" route add default via 10.10.10.1; done
  ip -n S route add default via 10.10.20.1
  for n in E E2; do ip -n "$n" route add default via 192.0.2.1; done
  ip netns exec G sysctl -qw net.ipv4.ip_forward=1
  # IPv6 (laboratorio 8): so se o kernel tiver suporte a IPv6
  if [ ! -d /proc/sys/net/ipv6 ]; then echo "Aviso: kernel sem IPv6; enderecos IPv6 nao configurados."
>&2; return 0; fi
  ip -n A addr add 2001:db8:10::10/64 dev eth0 nodad; ip -n G addr add 2001:db8:10::1/64 dev lan0 nodad
  ip -n S addr add 2001:db8:20::20/64 dev eth0 nodad; ip -n G addr add 2001:db8:20::1/64 dev dmz0 nodad
  ip -n A route add default via 2001:db8:10::1; ip -n S route add default via 2001:db8:20::1
  ip netns exec G sysctl -qw net.ipv6.conf.all.forwarding=1
}
"${1:-criar}"
```

lab-host-reject.nft e lab-gateway-input-errado.nft (os demais estão nos capítulos 22, 27, 30 e 31)

```
# Variante do laboratorio 2: o restante de TCP recebe reject (RST) em vez de drop
table inet lab_host {
  chain input {
    type filter hook input priority 0; policy drop;
    iifname "lo" accept
    ct state invalid drop
    ct state established,related accept
    meta l4proto { icmp, ipv6-icmp } accept
    ip saddr 10.10.10.10 tcp dport 8080 counter accept comment "LAB: cliente A"
    meta l4proto tcp counter reject with tcp reset
    counter drop
  }
}

# Erro recorrente: liberar INPUT no gateway quando o servico esta na DMZ
table inet lab_erro {
  chain input {
    type filter hook input priority 0; policy accept;
```

```
    tcp dport 8080 counter accept comment "LAB: regra no lugar errado"
  }
  chain forward {
    type filter hook forward priority 0; policy drop;
    counter comment "LAB: tudo que chega ao forward"
  }
}
```

executar_laboratorios.sh

```
#!/usr/bin/env bash
# Executa os laboratorios Linux do manual numa topologia de namespaces (topologia.sh).
# Somente em maquina de laboratorio: cria namespaces, carrega nftables neles e liga
# o log de netfilter para namespaces (net.netfilter.nf_log_all_netns), restaurado ao final.
# Uso: sudo ./executar_laboratorios.sh <pasta_saidas> <pasta_capturas>
set -u
AQUI="$(cd "$(dirname "$0")" && pwd)"
O="$(realpath -m "${1:-saidas}")"; C="$(realpath -m "${2:-capturas}")"; mkdir -p "$O" "$C"
WEB="$(mktemp -d)"; printf 'LAB FIREWALL OK\n' > "$WEB/index.html"
N() { ip netns exec "$@"; }
LOG_ANTES=$(cat /proc/sys/net/netfilter/nf_log_all_netns)
sysctl -qw net.netfilter.nf_log_all_netns=1
cmd() { echo "\$ $*"; }

teste() {  # teste <ns> <url> [rotulo]
  local r; r=$(N "$1" curl -s --noproxy '*' --max-time 3 -o /dev/null -w '%{http_code} %{time_total}'
"$2" 2>/dev/null); local rc=$?
  printf '%-4s %-34s -> HTTP %s  tempo %ss  curl_exit=%s\n' "$1" "$2" "${r% *}" "${r#* }" "$rc"
}
servidor() {  # servidor <porta> : HTTP didatico em S; espera o listener aparecer
  ip netns exec S python3 -m http.server "$1" --bind 10.10.20.20 --directory "$WEB" >/dev/null 2>&1 &
  echo $! >> "$WEB/pids"
  for _ in $(seq 1 50); do N S ss -lnt | grep -q "10.10.20.20:$1 " && return 0; sleep 0.1; done
  echo "listener $1 nao subiu" >&2
}
para_servidores() { [ -f "$WEB/pids" ] && kill $(cat "$WEB/pids") 2>/dev/null; rm -f "$WEB/pids"; sleep
0.5; }
captura() {  # captura <ns> <if> <arquivo> <segundos>  (em segundo plano; chame 'pronto' depois)
  ip netns exec "$1" dumpcap -q -i "$2" -a duration:"$4" -w "$C/$3" >/dev/null 2>&1 &
}
pronto() { sleep 1.5; }
resumo() { tshark -n -r "$C/$1" -t ud -o tcp.relative_sequence_numbers:FALSE 2>/dev/null | sed 's/^
*//' | cut -c1-118; }
limpa_nft() { for n in G S; do N "$n" nft flush ruleset; done; }
dmesg_lab() { dmesg | grep "$1" | tail -n "${2:-3}" | sed -E 's/^\[[ 0-9.]+\] //' | cut -c1-150; }

"$AQUI/topologia.sh" criar 2>"$O/00_topologia_aviso.txt"

# ------------------------------------------------------------------ 00 topologia
{ echo "Kernel: $(uname -r)  nftables: $(nft --version)  conntrack: $(conntrack --version 2>&1 | head
-1)"
  for n in A B G S E E2 R; do printf '%-3s ' "$n"; N "$n" ip -br -4 addr | grep -v '^lo' | awk '{printf
"%s %s  ", $1, $3}'; echo; done
  cat "$O/00_topologia_aviso.txt"; } > "$O/00_topologia.txt"

# ------------------------------------------------------------------ 01 linha de base
servidor 8080
{ cmd "ss -lnt   (em S)"; N S ss -lnt | sed -n '1p;/:8080/p'
  cmd "curl local em S e a partir de A e B, sem nenhuma regra"
  teste S http://10.10.20.20:8080/; teste A http://10.10.20.20:8080/; teste B http://
10.10.20.20:8080/; } > "$O/01_linha_base.txt"

# ------------------------------------------------------------------ 02 politica de host em S
N S nft -c -f "$AQUI/lab-host.nft" && N S nft -f "$AQUI/lab-host.nft"
```

```
captura S eth0 02_S_eth0_B_drop.pcapng 7; pronto
{ cmd "nft -c -f lab-host.nft && nft -f lab-host.nft   (em S)"
  teste A http://10.10.20.20:8080/; teste B http://10.10.20.20:8080/; sleep 3
  echo; cmd "nft list table inet lab_host   (contadores)"; N S nft list table inet lab_host | grep -E
'counter|policy'
  echo; cmd "dmesg | grep LAB_HOST_DROP"; dmesg_lab LAB_HOST_DROP 2
  echo; cmd "tshark -r 02_S_eth0_B_drop.pcapng   (captura em S durante as tentativas)"; resumo
02_S_eth0_B_drop.pcapng | grep -E '10\.10\.10\.11' | head -6
  N S nft delete table inet lab_host
  echo; cmd "nft delete table inet lab_host  -> nova tentativa"; teste B http://10.10.20.20:8080/; } >
"$O/02_host_linux.txt"

# ------------------------------------------------------------------ 03 drop x reject
{ cmd "B contra S com a politica que DESCARTA (lab-host.nft)"
  N S nft -f "$AQUI/lab-host.nft"; teste B http://10.10.20.20:8080/; N S nft delete table inet lab_host
  cmd "B contra S com a politica que REJEITA com RST (lab-host-reject.nft)"
  N S nft -f "$AQUI/lab-host-reject.nft"; captura S eth0 03_S_reject.pcapng 3; pronto
  teste B http://10.10.20.20:8080/; sleep 2.5
  resumo 03_S_reject.pcapng | grep -E '10\.10\.10\.11' | head -3
  N S nft delete table inet lab_host
  echo; cmd "B contra S SEM firewall e SEM servico na porta 9999"
  teste B http://10.10.20.20:9999/; } > "$O/03_drop_reject.txt"

# ------------------------------------------------------------------ 04 input x forward no gateway
{ cmd "nft -f lab-gateway-input-errado.nft   (em G: libera INPUT 8080, forward drop)"
  N G nft -f "$AQUI/lab-gateway-input-errado.nft"; teste A http://10.10.20.20:8080/
  N G nft list table inet lab_erro | grep counter
  N G nft delete table inet lab_erro
  echo; cmd "nft -c -f lab-router.nft && nft -f lab-router.nft   (em G)"
  N G nft -c -f "$AQUI/lab-router.nft" && N G nft -f "$AQUI/lab-router.nft"
  captura G lan0 04_G_lan0.pcapng 7; captura G dmz0 04_G_dmz0.pcapng 7; pronto
  teste A http://10.10.20.20:8080/; teste B http://10.10.20.20:8080/; sleep 3
  echo; cmd "contadores de lab_router"; N G nft list table inet lab_router | grep counter
  echo; cmd "dmesg | grep LAB_FWD_DROP"; dmesg_lab LAB_FWD_DROP 2
  echo; cmd "SYN de B (10.10.10.11) em cada interface de G"
  printf 'lan0: %s quadro(s)   dmz0: %s quadro(s)\n' \
    "$(tshark -n -r "$C/04_G_lan0.pcapng" -Y 'ip.src==10.10.10.11 && tcp.flags.syn==1' 2>/dev/null | wc
-l)" \
    "$(tshark -n -r "$C/04_G_dmz0.pcapng" -Y 'ip.src==10.10.10.11 && tcp.flags.syn==1' 2>/dev/null | wc
-l)"
  printf 'SYN de A: lan0 %s, dmz0 %s\n' \
    "$(tshark -n -r "$C/04_G_lan0.pcapng" -Y 'ip.src==10.10.10.10 && tcp.flags.syn==1 &&
tcp.flags.ack==0' 2>/dev/null | wc -l)" \
    "$(tshark -n -r "$C/04_G_dmz0.pcapng" -Y 'ip.src==10.10.10.10 && tcp.flags.syn==1 &&
tcp.flags.ack==0' 2>/dev/null | wc -l)"; } > "$O/04_forward.txt"

# ------------------------------------------------------------------ 05 rastreamento
{ cmd "nft -f lab-trace.nft ; nft monitor trace   (em G, durante uma tentativa de B)"
  N G nft -f "$AQUI/lab-trace.nft"
  (N G timeout 4 nft monitor trace > "$WEB/trace.txt" 2>&1 &) ; sleep 1
  N B curl -s --max-time 1 -o /dev/null http://10.10.20.20:8080/; sleep 3
  awk '!visto[$0]++' "$WEB/trace.txt" | sed -E 's/trace id [0-9a-f]+ /trace id … /' | cut -c1-150 |
head -12
  N G nft delete table ip lab_trace; } > "$O/05_trace.txt"

# ------------------------------------------------------------------ 06 estados: sessao antiga x nova
cat > "$WEB/cliente.py" <<'PY'
import socket, time, sys
s = socket.create_connection(("10.10.20.20", 8080), 3)
print("conectado de", s.getsockname(), flush=True)
time.sleep(float(sys.argv[1]))
try:
    s.sendall(b"GET / HTTP/1.0\r\nHost: lab\r\n\r\n")
    print("resposta:", s.recv(200).split(b"\r\n")[0].decode(), flush=True)
except OSError as e:
```

```
    print("erro:", type(e).__name__, flush=True)
s.close()
PY
{ cmd "A abre uma conexao TCP e fica ociosa (python cliente.py 6)"
  N A python3 "$WEB/cliente.py" 6 > "$WEB/sessao.txt" 2>&1 & PID=$!; sleep 1
  cmd "conntrack -L -p tcp --dport 8080   (em G)"; N G conntrack -L -p tcp --dport 8080 2>/dev/null |
sed -E 's/ mark=0.*//'
  H=$(N G nft -a list chain inet lab_router forward | awk '/LAB: A para S:8080/{print $NF}')
  cmd "nft delete rule inet lab_router forward handle $H   (remove a permissao de A)"
  N G nft delete rule inet lab_router forward handle "$H"
  cmd "nova conexao de A"; teste A http://10.10.20.20:8080/
  wait $PID; cmd "a conexao antiga envia o pedido depois da remocao"; cat "$WEB/sessao.txt"
  N G nft delete table inet lab_router; N G nft -f "$AQUI/lab-router.nft"   # recarga limpa: nft -f
sobre tabela existente acrescenta regras
  echo; cmd "variante: remover tambem o estado do fluxo (conntrack -D) antes do pedido"
  N A python3 "$WEB/cliente.py" 4 > "$WEB/sessao2.txt" 2>&1 & PID=$!; sleep 1
  H=$(N G nft -a list chain inet lab_router forward | awk '/LAB: A para S:8080/{print $NF}')
  N G nft delete rule inet lab_router forward handle "$H"
  cmd "conntrack -D -p tcp -s 10.10.10.10 --dport 8080"
  N G conntrack -D -p tcp -s 10.10.10.10 --dport 8080 2>&1 | grep -v '^tcp' | sed 's/^/  /'
  wait $PID; cat "$WEB/sessao2.txt"
  N G nft delete table inet lab_router; } > "$O/06_estados.txt"

# ------------------------------------------------------------------ 07 DNAT e retorno
N G nft -c -f "$AQUI/lab-nat.nft" && N G nft -f "$AQUI/lab-nat.nft"
captura G wan0 07_G_wan0.pcapng 7; captura G dmz0 07_G_dmz0.pcapng 7; pronto
{ cmd "nft -f lab-nat.nft   (em G)"
  teste E http://192.0.2.1:18080/; teste E2 http://192.0.2.1:18080/; sleep 3
  echo; cmd "conntrack -L -p tcp --dport 18080   (em G: tupla original e de retorno)"
  N G conntrack -L -p tcp --orig-port-dst 18080 2>/dev/null | sed -E 's/ mark=0.*//' | head -2
  echo; cmd "a mesma conversa de E vista em wan0 e dmz0 (primeiros quadros)"
  echo "wan0:"; resumo 07_G_wan0.pcapng | grep '192.0.2.10 ' | head -2
  echo "dmz0:"; resumo 07_G_dmz0.pcapng | grep '192.0.2.10 ' | head -2
  echo; cmd "contadores"; N G nft list ruleset | grep -E 'dnat to|counter packets' | sed 's/^\s*//'; }
> "$O/07_dnat.txt"

# ------------------------------------------------------------------ 08 caso 2: retorno por outro
roteador
N S ip route replace default via 10.10.20.254
captura S eth0 08_S_eth0.pcapng 6; captura G dmz0 08_G_dmz0.pcapng 6; pronto
{ cmd "ip route replace default via 10.10.20.254   (em S: gateway errado R)"
  teste E http://192.0.2.1:18080/; sleep 3
  printf 'Em S:     SYN de E recebidos %s, SYN/ACK enviados %s\n' \
    "$(tshark -n -r "$C/08_S_eth0.pcapng" -Y 'ip.src==192.0.2.10 && tcp.flags.syn==1 &&
tcp.flags.ack==0' 2>/dev/null | wc -l)" \
    "$(tshark -n -r "$C/08_S_eth0.pcapng" -Y 'ip.dst==192.0.2.10 && tcp.flags.syn==1 &&
tcp.flags.ack==1' 2>/dev/null | wc -l)"
  printf 'Em G/dmz0: SYN/ACK para E vistos %s\n' \
    "$(tshark -n -r "$C/08_G_dmz0.pcapng" -Y 'ip.dst==192.0.2.10 && tcp.flags.syn==1 &&
tcp.flags.ack==1' 2>/dev/null | wc -l)"
  echo "Destino L2 do SYN/ACK em S (MAC de R x MAC de G):"
  printf '  MAC de G dmz0: %s   MAC de R: %s\n' "$(N G cat /sys/class/net/dmz0/address)" "$(N R cat /
sys/class/net/eth0/address)"
  tshark -n -r "$C/08_S_eth0.pcapng" -Y 'ip.dst==192.0.2.10 && tcp.flags.syn==1 && tcp.flags.ack==1' -T
fields -e eth.dst 2>/dev/null | sort | uniq -c | sed 's/^ */  SYN\/ACK para /'
  N S ip route replace default via 10.10.20.1
  echo; cmd "ip route replace default via 10.10.20.1   (corrigido)"; teste E http://192.0.2.1:18080/; }
> "$O/08_caso_retorno.txt"
N G nft delete table ip lab_nat; N G nft delete table inet lab_pub

# ------------------------------------------------------------------ 09 linha do tempo (RST sem listener)
para_servidores
captura A eth0 09_A.pcapng 5; captura G lan0 09_G_lan0.pcapng 5; captura G dmz0 09_G_dmz0.pcapng 5;
captura S eth0 09_S.pcapng 5; pronto
{ cmd "servidor parado; A tenta S:8080 com 4 pontos de captura"; teste A http://10.10.20.20:8080/;
```

```
sleep 3
  for f in 09_A.pcapng:A 09_G_lan0.pcapng:G/lan0 09_G_dmz0.pcapng:G/dmz0 09_S.pcapng:S; do
    tshark -n -r "$C/${f%%:*}" -t ud -Y 'tcp.port==8080' -T fields -e frame.time_utc -e ip.src -e
ip.dst -e tcp.flags.str 2>/dev/null \
      | awk -v p="${f##*:}" -F'\t' '{split($1,a," "); printf "%s UTC  %-7s %s -> %s  %s\n", a[4], p,
$2, $3, $4}'
  done | sort | sed -E 's/^([0-9:]+\.[0-9]{6})[0-9]*/\1/'; } > "$O/09_linha_tempo.txt"

# ------------------------------------------------------------------ 10 mudanca 8080 -> 8082 com reversao
servidor 8080; servidor 8082
sed 's/tcp dport 8080 counter accept comment "LAB: A para S:8080"/tcp dport 8082 counter accept comment
"LAB: A para S:8082"/' "$AQUI/lab-router.nft" > "$WEB/lab-router-8082.nft"
{ cmd "1. linha de base (lab-router.nft, 8080)"; N G nft -f "$AQUI/lab-router.nft"
  teste A http://10.10.20.20:8080/; teste B http://10.10.20.20:8080/
  cmd "2-4. troca pela regra 8082 (arquivo novo validado com nft -c)"
  N G nft -c -f "$WEB/lab-router-8082.nft" && N G nft delete table inet lab_router && N G nft -f "$WEB/
lab-router-8082.nft"
  teste A http://10.10.20.20:8082/; teste B http://10.10.20.20:8082/; teste A http://10.10.20.20:8080/
  cmd "5-6. reversao: volta lab-router.nft e repete os testes"
  N G nft delete table inet lab_router && N G nft -f "$AQUI/lab-router.nft"
  teste A http://10.10.20.20:8080/; teste B http://10.10.20.20:8080/; teste A http://
10.10.20.20:8082/; } > "$O/10_mudanca.txt"

# ------------------------------------------------------------------ limpeza
para_servidores; limpa_nft
"$AQUI/topologia.sh" destruir
sysctl -qw net.netfilter.nf_log_all_netns="$LOG_ANTES"
(cd "$C" && sha256sum *.pcapng) > "$O/99_capturas_sha256.txt"
rm -rf "$WEB"
echo "Saidas em $O; capturas em $C"
```

EXECUTADO SHA256 das capturas reais anexadas:

```
b574cc91cdfd0fcfc2735298ac57fb40817fea8ffa75645ca72f80ee7ae7adf9  02_S_eth0_B_drop.pcapng
a1bc634a2589050b570cf843686af33b718e4fd2ff3b15d92378c89d42d52007  03_S_reject.pcapng
e638d2cfd3c9a16fe3588a6f9f8374b3885564a9bd26e5650a3d61de1c856482  04_G_dmz0.pcapng
3a6d738d2422647b7beac58bd74719c76865068aaa35accef4efb458c31d8b99  04_G_lan0.pcapng
3b3e018c353767a73c59caa86cbccdfd02869f33891df91626395da616bcb92c  07_G_dmz0.pcapng
9589a0efed72ee82eb314819d3ab64a6a937ea772fcaf919f9bae8214780a9dd  07_G_wan0.pcapng
c3ebe2e8d1447f9856d5e84c0bbf7426971f419f0b7ffafe6837d6d1486eb716  08_G_dmz0.pcapng
1da6a5ba5700abf7bae4801fc49edabccd969cda428d8e53caa21f25fad5af8a  08_S_eth0.pcapng
1e714b03c5e8fd7e0e3cf9be464068afc286b11ec4ee2f9caabfea1fe02436c8  09_A.pcapng
c3f65a45df4994e81df96028a5517de2f49071ae06f6915ae2be6ed4d29c884b  09_G_dmz0.pcapng
a92f728a6ba350dd4e75dfbad887158762cdbb167fd545df279eede604483a41  09_G_lan0.pcapng
e686de27b38d00d9fcc197901d028df59265de6f2b9dad2ef851ce795c1f4114  09_S.pcapng
```

E.2 Windows (NÃO EXECUTADO, sintaxe validada)

EXECUTADO Validação com a gramática tree-sitter-powershell e controle negativo (os scripts usam só ASCII, porque o Windows PowerShell 5.1 lê arquivos sem BOM como ANSI). No Windows, rode primeiro `validar_sintaxe.ps1`, que usa o parser oficial.

```
tree-sitter 0.26.0 / tree-sitter-language-pack 1.22.1 (gramatica powershell)
[OK]   evidencias_wfp.ps1              78 linhas, sem nos de erro, ASCII
[OK]   inventario_firewall.ps1         84 linhas, sem nos de erro, ASCII
[OK]   lab3_regra.ps1                  48 linhas, sem nos de erro, ASCII
[OK]   validar_sintaxe.ps1             30 linhas, sem nos de erro, ASCII
Resultado: 0 arquivo(s) com problema

tree-sitter 0.26.0 / tree-sitter-language-pack 1.22.1 (gramatica powershell)
[ERRO] neg.ps1
       erro na linha 2, coluna 3: b'param([string]$a\n  $a'
Resultado: 1 arquivo(s) com problema
```

POWERSHELL NÃO EXECUTADO inventario_firewall.ps1

```
<#
.SYNOPSIS
  Inventario do Windows Firewall para uma porta (somente leitura).
.DESCRIPTION
  1. Perfil de cada interface e configuracao de cada perfil (padroes e log).
  2. Regras HABILITADAS da politica efetiva (ActiveStore) que tocam a porta informada,
     com filtros de porta, endereco e programa, e marcando bloqueios explicitos,
     que prevalecem sobre permissoes conflitantes.
  3. Ultimas linhas do log do perfil, lidas pelo cabecalho #Fields do proprio arquivo.
  Exige sessao elevada para ler o log. ESTADO NESTA EDICAO: NAO EXECUTADO (sintaxe validada).
.EXAMPLE
  .\inventario_firewall.ps1 -Porta 8080 -Perfil Private
#>
[CmdletBinding()]
param(
    [Parameter(Mandatory = $true)]
    [ValidateRange(1, 65535)]
    [int]$Porta,

    [ValidateSet('Domain', 'Private', 'Public')]
    [string]$Perfil = 'Private',

    [int]$LinhasLog = 20
)

Write-Output '=== 1. Perfis ==='
Get-NetConnectionProfile | Format-Table -AutoSize -Property @('InterfaceAlias', 'NetworkCategory',
'IPv4Connectivity')
Get-NetFirewallProfile -PolicyStore ActiveStore |
    Format-Table -AutoSize -Property @('Name', 'Enabled', 'DefaultInboundAction',
'DefaultOutboundAction', 'LogAllowed', 'LogBlocked', 'LogFileName')

Write-Output ("=== 2. Regras habilitadas na politica efetiva que tocam a porta {0} ===" -f $Porta)
$regras = @(Get-NetFirewallRule -PolicyStore ActiveStore | Where-Object { $_.Enabled -eq 'True' })
$achadas = foreach ($r in $regras) {
    $pf = $r | Get-NetFirewallPortFilter
    $portas = @($pf.LocalPort) + @($pf.RemotePort)
    $toca = ($portas -contains 'Any') -or ($portas -contains [string]$Porta) -or
            (@($portas | Where-Object { $_ -match '^(\d+)-(\d+)$' -and [int]$Matches[1] -le $Porta -and
[int]$Matches[2] -ge $Porta }).Count -gt 0)
    if (-not $toca -or ($pf.Protocol -notin @('TCP', 'Any'))) { continue }
    $af = $r | Get-NetFirewallAddressFilter
    $ap = $r | Get-NetFirewallApplicationFilter
    [pscustomobject]@{
        Nome      = $r.Name
        Direcao   = [string]$r.Direction
        Acao      = [string]$r.Action
        Perfil    = [string]$r.Profile
        Protocolo = $pf.Protocol
```

```
        PortaLoc  = ($pf.LocalPort -join ',')
        Remoto    = ($af.RemoteAddress -join ',')
        Programa  = $ap.Program
        Origem    = [string]$r.PolicyStoreSourceType
    }
}
$achadas = @($achadas)
$achadas | Sort-Object -Property @('Direcao', 'Acao') | Format-Table -AutoSize -Wrap
$bloqueios = @($achadas | Where-Object { $_.Acao -eq 'Block' })
if ($bloqueios.Count -gt 0) {
    Write-Output 'ATENCAO: ha bloqueio explicito para esta porta; ele prevalece sobre permissoes
conflitantes.'
}
$amplas = @($achadas | Where-Object { $_.Acao -eq 'Allow' -and $_.Remoto -eq 'Any' })
if ($amplas.Count -gt 0) {
    Write-Output 'ATENCAO: ha permissao sem restricao de origem; um teste negativo pode nao provar a
restricao pretendida.'
}

Write-Output ''
Write-Output ("=== 3. Log do perfil {0} ===" -f $Perfil)
$p = Get-NetFirewallProfile -PolicyStore ActiveStore -Name $Perfil
$arquivo = [Environment]::ExpandEnvironmentVariables([string]$p.LogFileName)
if (-not (Test-Path -LiteralPath $arquivo)) {
    Write-Output ("Log nao encontrado em {0}. LogBlocked={1} LogAllowed={2}" -f $arquivo,
$p.LogBlocked, $p.LogAllowed)
    exit 0
}
$linhas = Get-Content -LiteralPath $arquivo -Tail 2000
$cab = $linhas | Where-Object { $_ -like '#Fields:*' } | Select-Object -Last 1
if (-not $cab) { $cab = (Get-Content -LiteralPath $arquivo -TotalCount 10 | Where-Object { $_ -like
'#Fields:*' } | Select-Object -First 1) }
$campos = @(($cab -replace '^#Fields:\s*', '') -split '\s+')
$dados = $linhas | Where-Object { $_ -and $_ -notlike '#*' } | ForEach-Object {
    $v = $_ -split '\s+'
    $o = [ordered]@{}
    for ($i = 0; $i -lt $campos.Count; $i++) { $o[$campos[$i]] = $(if ($i -lt $v.Count) { $v[$i] } else
{ '' }) }
    [pscustomobject]$o
}
$dados | Where-Object { $_.'dst-port' -eq [string]$Porta -or $_.'src-port' -eq [string]$Porta } |
    Select-Object -Last $LinhasLog | Format-Table -AutoSize
Write-Output ("Campos do cabecalho: {0}" -f ($campos -join ' '))
```

POWERSHELL NÃO EXECUTADO evidencias_wfp.ps1

```
<#
.SYNOPSIS
  Evidencia de bloqueio pelo Windows Filtering Platform (somente leitura, exceto o XML de filtros).
.DESCRIPTION
  1. Verifica a auditoria das subcategorias Filtering Platform Connection e Packet Drop (por GUID).
  2. Lista eventos 5157 (conexao bloqueada) e 5152 (pacote bloqueado) recentes para a porta.
  3. Gera filters.xml com 'netsh wfp show filters' numa pasta nova e localiza o filtro
     cujo filterId corresponde ao Filter Run-Time ID do evento.
  Exige sessao elevada. ESTADO NESTA EDICAO: NAO EXECUTADO (sintaxe validada).
.EXAMPLE
  .\evidencias_wfp.ps1 -Porta 8080 -Minutos 30 -Pasta C:\Lab\wfp
#>
[CmdletBinding()]
param(
    [Parameter(Mandatory = $true)]
    [int]$Porta,
    [int]$Minutos = 30,
    [string]$Pasta = 'C:\Lab\wfp'
```

```
)

$subs = @(
    @{ Nome = 'Filtering Platform Packet Drop'; Guid = '{0CCE9225-69AE-11D9-BED3-505054503030}';
Eventos = '5152' },
    @{ Nome = 'Filtering Platform Connection';  Guid = '{0CCE9226-69AE-11D9-BED3-505054503030}';
Eventos = '5156, 5157' }
)
Write-Output '=== 1. Auditoria do WFP ==='
foreach ($s in $subs) {
    $saida = & auditpol.exe /get ('/subcategory:' + $s.Guid) /r 2>&1
    $linha = @($saida | Where-Object { $_ -match [regex]::Escape($s.Guid) }) | Select-Object -First 1
    $config = ''
    if ($linha) { $config = (($linha -split ',')[4]) }
    Write-Output ("{0,-32} eventos {1,-11} configuracao: {2}" -f $s.Nome, $s.Eventos, $config)
}
Write-Output 'Sem auditoria, a ausencia de 5157/5152 nao prova que nada foi bloqueado.'

Write-Output ''
Write-Output ("=== 2. Eventos 5157/5152 dos ultimos {0} min com a porta {1} ===" -f $Minutos, $Porta)
$eventos = @()
try {
    $eventos = @(Get-WinEvent -FilterHashtable @{ LogName = 'Security'; Id = @(5157, 5152); StartTime =
(Get-Date).AddMinutes(-$Minutos) } -ErrorAction Stop)
} catch {
    if ($_.FullyQualifiedErrorId -notlike 'NoMatchingEventsFound*') { Write-Error $_.Exception.Message;
exit 3 }
}
$linhas = foreach ($e in $eventos) {
    [xml]$x = $e.ToXml()
    $d = @{}
    foreach ($i in $x.Event.EventData.Data) { $d[[string]$i.Name] = [string]$i.'#text' }
    if ($d['SourcePort'] -ne [string]$Porta -and $d['DestPort'] -ne [string]$Porta) { continue }
    [pscustomobject]@{
        UTC       = $e.TimeCreated.ToUniversalTime().ToString('s')
        ID        = $e.Id
        Direcao   = $d['Direction']   # vem como codigo %%NNNNN; o texto aparece em Message
        Origem    = ('{0}:{1}' -f $d['SourceAddress'], $d['SourcePort'])
        Destino   = ('{0}:{1}' -f $d['DestAddress'], $d['DestPort'])
        Protocolo = $d['Protocol']
        FiltroId  = $d['FilterRTID']
        Aplicacao = $d['Application']
    }
}
$linhas = @($linhas)
$linhas | Format-Table -AutoSize -Wrap
if ($linhas.Count -eq 0) { Write-Output 'Nenhum evento correspondente.'; exit 0 }

Write-Output '=== 3. Filtro WFP correspondente ==='
New-Item -ItemType Directory -Path $Pasta -Force | Out-Null
# 'netsh wfp show filters' grava filters.xml na pasta atual (pagina do evento 5157)
Push-Location -LiteralPath $Pasta
& netsh.exe wfp show filters | Out-Null
Pop-Location
$xml = Join-Path -Path $Pasta -ChildPath 'filters.xml'
if (-not (Test-Path -LiteralPath $xml)) { Write-Output 'netsh wfp show filters nao gerou o arquivo.';
exit 1 }
[xml]$f = Get-Content -LiteralPath $xml -Raw
$ids = @($linhas | ForEach-Object { $_.FiltroId } | Select-Object -Unique)
foreach ($id in $ids) {
    $no = $f.SelectNodes('//item') | Where-Object { $_.filterId -eq $id } | Select-Object -First 1
    if ($null -eq $no) { Write-Output ("filterId {0}: nao encontrado (o filtro pode ter mudado desde o
evento)" -f $id); continue }
    Write-Output ("filterId {0}: {1} | {2}" -f $id, $no.displayData.name, $no.displayData.description)
```

```
}
Write-Output ("Arquivo de filtros preservado em {0}" -f $xml)
```

POWERSHELL NÃO EXECUTADO lab3_regra.ps1 (MUDANÇA, só em VM de laboratório)

```
<#
.SYNOPSIS
  Laboratorio 3 (MUDANCA, so em VM de laboratorio): cria ou remove LAB-HTTP-A e o log do perfil.
.DESCRIPTION
  -Criar: grava os valores atuais de log do perfil em estado_log.json, cria a regra
          LAB-HTTP-A (entrada, TCP 8080, origem 10.10.10.10) e liga o log de bloqueios e permissoes.
  -Remover: remove somente LAB-HTTP-A e restaura os valores de log gravados.
  ESTADO NESTA EDICAO: NAO EXECUTADO (sintaxe validada).
.EXAMPLE
  .\lab3_regra.ps1 -Criar
  .\lab3_regra.ps1 -Remover
#>
[CmdletBinding()]
param(
    [Parameter(Mandatory = $true, ParameterSetName = 'Criar')]
    [switch]$Criar,
    [Parameter(Mandatory = $true, ParameterSetName = 'Remover')]
    [switch]$Remover,
    [string]$Perfil = 'Private',
    [string]$Origem = '10.10.10.10',
    [int]$Porta = 8080
)

$estado = Join-Path -Path $PSScriptRoot -ChildPath 'estado_log.json'

if ($Criar) {
    if (Get-NetFirewallRule -Name 'LAB-HTTP-A' -ErrorAction SilentlyContinue) { Write-Error 'LAB-HTTP-A
ja existe.'; exit 3 }
    $p = Get-NetFirewallProfile -Name $Perfil
    [pscustomobject]@{ Perfil = $Perfil; LogAllowed = [string]$p.LogAllowed; LogBlocked = [string]
$p.LogBlocked } |
        ConvertTo-Json | Set-Content -LiteralPath $estado -Encoding UTF8
    New-NetFirewallRule -Name 'LAB-HTTP-A' -DisplayName 'LAB HTTP somente cliente A' `
        -Direction Inbound -Action Allow -Profile $Perfil -Protocol TCP -LocalPort $Porta -
RemoteAddress $Origem | Out-Null
    Set-NetFirewallProfile -Name $Perfil -LogBlocked True -LogAllowed True
    Write-Output ("LAB-HTTP-A criada; valores anteriores de log em {0}" -f $estado)
    exit 0
}

if ($Remover) {
    Remove-NetFirewallRule -Name 'LAB-HTTP-A' -ErrorAction SilentlyContinue
    if (Test-Path -LiteralPath $estado) {
        $a = Get-Content -LiteralPath $estado -Raw | ConvertFrom-Json
        Set-NetFirewallProfile -Name $a.Perfil -LogAllowed $a.LogAllowed -LogBlocked $a.LogBlocked
        Remove-Item -LiteralPath $estado
        Write-Output ("LAB-HTTP-A removida; log do perfil {0} restaurado (LogAllowed={1},
LogBlocked={2})" -f $a.Perfil, $a.LogAllowed, $a.LogBlocked)
    } else {
        Write-Output 'LAB-HTTP-A removida; estado_log.json nao encontrado, log nao restaurado.'
    }
}
```

F Referências oficiais

Consultadas em 30/10/2026. A referência NIST é histórica e fundamenta conceitos; procedimentos dependem da documentação e da versão da plataforma. Casos, topologias, matrizes e exercícios são sínteses didáticas. A Edição 1 só tinha os links clicáveis; agora as URLs estão impressas.

[1] NIST SP 800-41 Rev. 1 — Guidelines on Firewalls and Firewall Policy (2009)
https://csrc.nist.gov/pubs/sp/800/41/r1/final

[2] Netgate — Rule Methodology
https://docs.netgate.com/pfsense/en/latest/firewall/rule-methodology.html

[3] AWS — Security groups
https://docs.aws.amazon.com/vpc/latest/userguide/vpc-security-groups.html

[4] AWS — Network ACLs
https://docs.aws.amazon.com/vpc/latest/userguide/vpc-network-acls.html

[5] Netfilter — Matching connection tracking stateful metainformation
https://wiki.nftables.org/wiki-nftables/index.php/Matching_connection_tracking_stateful_metainformation

[6] RFC 1928 — SOCKS Protocol Version 5
https://www.rfc-editor.org/info/rfc1928/

[7] Palo Alto Networks — App-ID Overview
https://docs.paloaltonetworks.com/ngfw/administration/app-id/app-id-overview

[8] Palo Alto Networks — Safely Enable Applications on Default Ports
https://docs.paloaltonetworks.com/ngfw/administration/app-id/application-default

[9] AWS — What is AWS WAF?
https://docs.aws.amazon.com/waf/latest/developerguide/what-is-aws-waf.html

[10] Netfilter — Configuring chains
https://wiki.nftables.org/wiki-nftables/index.php/Configuring_chains

[11] Microsoft — Windows Firewall rules
https://learn.microsoft.com/en-us/windows/security/operating-system-security/network-security/windows-firewall/rules

[12] Netfilter — nft(8)
https://netfilter.org/projects/nftables/manpage.html

[13] Netgate — Viewing Firewall States in the GUI
https://docs.netgate.com/pfsense/en/latest/monitoring/status/firewall-states-gui.html

[14] RFC 9293 — TCP
https://www.rfc-editor.org/rfc/rfc9293.html

[15] Netfilter — Performing NAT
https://wiki.nftables.org/wiki-nftables/index.php/Performing_Network_Address_Translation_(NAT)

[16] Netgate — Ordering of NAT and Firewall Processing
https://docs.netgate.com/pfsense/en/latest/nat/process-order.html

[17] RFC 4890 — Filtering ICMPv6 Messages
https://www.rfc-editor.org/info/rfc4890/

[18] RFC 8201 — Path MTU Discovery for IPv6
https://www.rfc-editor.org/rfc/rfc8201.html

[19] Microsoft — Get-NetFirewallRule
https://learn.microsoft.com/en-us/powershell/module/netsecurity/get-netfirewallrule

[20] Microsoft — New-NetFirewallRule
https://learn.microsoft.com/en-us/powershell/module/netsecurity/new-netfirewallrule

[21] Microsoft — Configure Windows Firewall logging
https://learn.microsoft.com/en-us/windows/security/operating-system-security/network-security/windows-firewall/configure-logging

[22] Microsoft — Set-NetFirewallProfile
https://learn.microsoft.com/en-us/powershell/module/netsecurity/set-netfirewallprofile

[23] Canonical — nftables (Ubuntu security documentation)
https://documentation.ubuntu.com/security/security-features/network/firewall/nftables/

[24] Python — http.server
https://docs.python.org/3/library/http.server.html

[25] Netgate — Port Forwarding
https://docs.netgate.com/pfsense/en/latest/nat/port-forwards.html

[26] Wireshark — Filtering Packets While Viewing
https://www.wireshark.org/docs/wsug_html_chunked/ChWorkDisplayFilterSection.html
Link corrigido: faltava ".html" na Edição 1.

[27] Netgate — Troubleshooting NAT Port Forwards
https://docs.netgate.com/pfsense/en/latest/troubleshooting/nat-port-forwards.html

[28] Microsoft — Configure firewall for AD domains and trusts
https://learn.microsoft.com/en-us/troubleshoot/windows-server/active-directory/config-firewall-for-ad-domains-and-trusts

[29] Microsoft — Service overview and network port requirements
https://learn.microsoft.com/en-us/troubleshoot/windows-server/networking/service-overview-and-network-port-requirements

[30] Netgate — High Availability
https://docs.netgate.com/pfsense/en/latest/highavailability/index.html

[31] Microsoft — Evento 5157
https://learn.microsoft.com/en-us/previous-versions/windows/it-pro/windows-10/security/threat-protection/auditing/event-5157

[32] Microsoft — Evento 5152
https://learn.microsoft.com/en-us/previous-versions/windows/it-pro/windows-10/security/threat-protection/auditing/event-5152

[33] Microsoft — Evento 5156
https://learn.microsoft.com/en-us/previous-versions/windows/it-pro/windows-10/security/threat-protection/auditing/event-5156

[34] Microsoft — MS-GPAC, Subcategory e SubcategoryGUID
https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-gpac/77878370-0712-47cd-997d-b07053429f6d

[35] Netfilter — Ruleset debug/tracing; Logging traffic
https://wiki.nftables.org/wiki-nftables/index.php/Ruleset_debug/tracing
https://wiki.nftables.org/wiki-nftables/index.php/Logging_traffic

[36] Linux kernel — Netfilter sysfs variables (nf_log_all_netns)
https://docs.kernel.org/networking/netfilter-sysctl.html

[37] conntrack(8)
https://manpages.ubuntu.com/manpages/noble/man8/conntrack.8.html

[38] firewalld — firewall-cmd
https://firewalld.org/documentation/man-pages/firewall-cmd.html

[39] ufw(8)
https://manpages.ubuntu.com/manpages/noble/man8/ufw.8.html

[40] PyPI — tree-sitter-language-pack
https://pypi.org/project/tree-sitter-language-pack/
Ferramenta de terceiros, só para validar a sintaxe dos .ps1.

G Registro de edições e verificação

Edição	Data	Conteúdo
Edição 1	30/10/2026	Fundamentos, tipos, funcionamento, operação, nove laboratórios, quatro casos, projeto e apêndices. Nada executado.
Edição 2	30/10/2026	Laboratórios Linux executados em namespaces com saídas reais; capítulo de WFP; scripts Windows; roteiro pfSense; capturas anexadas; correções abaixo.

G.1 Correções

Ponto	Correção
Caminho do laboratório 3	`C:Labweb` (relativo, sem barras) passou a `C:\Lab\web`, igual ao laboratório 8.
Referência [26]	URL sem ".html" corrigida.
Referências	URLs impressas, não só clicáveis.
Linha do tempo e tabela de NAT	Previsões substituídas por observações reais (capítulos 15 e 32).

G.2 Acréscimos

Comparação executada de drop, reject e porta fechada; input × forward executado; precedência do Windows completada (especificidade entre permissões); conntrack e remoção delimitada de estados; nftrace com saída real; log de netfilter em namespaces; gerenciadores firewalld e UFW; aviso de que `nft -f` acrescenta regras; log do Windows Firewall (caminho, tamanho, permissão); eventos WFP 5157/5156/5152 e mapeamento do filtro; caso 2 reproduzido; laboratórios 4 e 6 com variante Linux; três exercícios novos.

G.3 Verificação

- Seis políticas nftables validadas com `nft -c -f` e carregadas; laboratórios 1, 2, 4, 5, 6, 7 e 9 e o caso 2 executados com nftables 1.0.9, conntrack 1.4.8 e Linux 6.18.
- 12 capturas reais com SHA256; contadores, log do kernel, conntrack e trace copiados da execução.
- Scripts PowerShell: sintaxe validada (tree-sitter) com controle negativo.

G.4 Pendências

- Executar os laboratórios 3 e 8 no Windows e os roteiros pfSense; rodar `validar_sintaxe.ps1`.
- Executar o laboratório 8 (IPv6) numa VM Linux com IPv6 no kernel, com a mesma topologia.
- Conferir no pfSense da versão do laboratório os nomes de menu citados.